



ORACLE
NetSuite

SuiteCommerce API Documentation

2025.2

September 3, 2025



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

APIs for SuiteCommerce Extensions	1
SuiteCommerce Extensibility API	1
Extensibility API Components	2
Instantiating API Components	11
Use SuiteScript With Your Extension	13
Item Search API	15
Item Search API	15
Items Available to the Item Search API	16
The Base URL	17
Item Search API Input Parameters	19
Item Search API Output Response	28
Sample Item Search Query and Results	31
Commerce API	36
Commerce API	36
Shopping Objects	37
Input Parameters/Return Values for Shopping Object Methods	37
ShoppingSession Methods	38
Customer Methods	64
Order Methods	76
Understanding Multiple Ship To	108
Working with Taxes using Commerce API	109
PageGenerator Methods	110
StandardTagLibrary Methods	113
JSON Object Fields	115
order	115
ach	116
address	116
analyticssettings	117
checkoutsettings	118
creditcard	119
customer	120
discounts_impact	122
discounts	123
fulfillmentPreferences	124
giftcertificate	124
item	125
message	126
orderitem	126
ordersettings	128
ordersummary	128
payment	130
paymentmethod	133
promocode	134
registrationsettings	135
shipmethod	136
sitecategory	136
sitesettings	137
status	139
subscriptions	140
summarytax	140
taxdetails	141
threedsecure	142
touchpoints	142

Serversync Touchpoint 143

SSP Application Governance 145

APIs for SuiteCommerce Extensions

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

When developing extensions for a SuiteCommerce website, you are likely to use one or more APIs to complete your customization. NetSuite and the SuiteCommerce framework provide the following APIs:

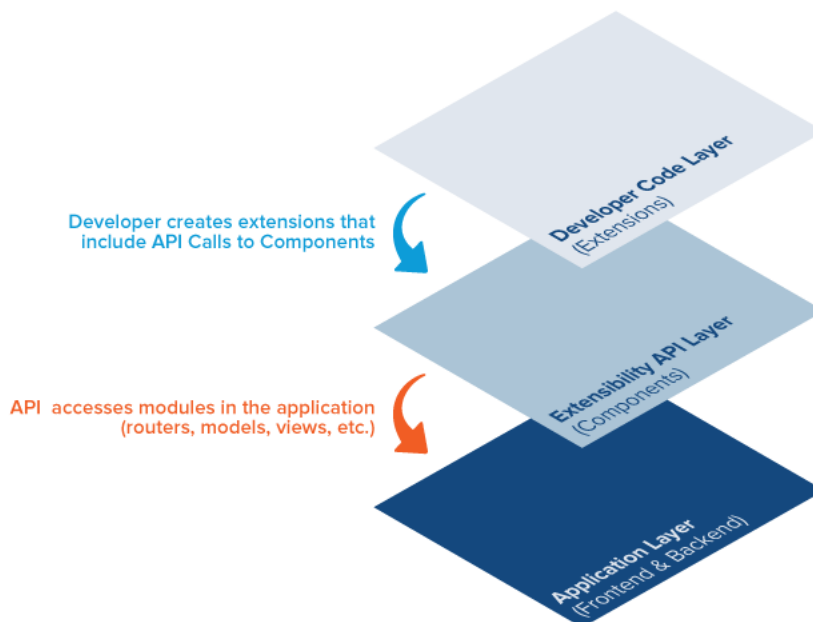
- **SuiteCommerce Extensibility API** – Use the extensibility API to interact with the application layer of a SuiteCommerce website. The extensibility API consists of several components that provide access to different parts of the SuiteCommerce application. For more information, see [SuiteCommerce Extensibility API](#).
- **SuiteScript API** – You can use the SuiteScript API to access custom records and NetSuite records, get and set record data, and perform record actions. SuiteScript 2.x is a JavaScript API that enables you to customize, extend, and process data in NetSuite. For more information, see [Use SuiteScript With Your Extension](#).
- **Item Search API** – The item search API is a REST style API that lets you to retrieve information from item records. It returns the information in JSON format. For more information, see [Item Search API](#).
- **Commerce API** – The Commerce API exposes a set of shopping objects and functions that you can use to develop web store customizations. For more information, see [Commerce API](#).

SuiteCommerce Extensibility API

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Note: This information refers to the SuiteCommerce and the Aconcagua release of SuiteCommerce Advanced and later.

SuiteCommerce includes a conceptual layer on top of existing SuiteCommerce architecture. This layer lets extension developers customize a Commerce web store and SuiteCommerce InStore (SCIS) with extensions, so that they don't have to change or customize the underlying application code.



Developers build extensions in their local environment with the extension developer tools. This environment/workspace is represented by the Developer Code Layer in the image above. Each extension can include JavaScript, SuiteScript, or configuration files to do its job. In this code, developers include API calls to components in the Extensibility API. This is represented by the Extensibility API Layer.

The API accesses frontend and backend modules, such as routers, models, and views, in the Application Layer. This way, extensions are separated programmatically from the main application and interact only with the API, not with the application. This makes customization easier and seamless for extension developers and ensures that future updates to the application do not compromise customizations.

See the following topics for more information about the Extensibility API:

- [Extensibility API Components](#) – This topic provides an overview of the Component Classes available for creating extensions.
- [Instantiating API Components](#) – This topic describes how to access specific components, such as Product Details Page or Product List Page.
- [SCIS Extensibility API](#) – This topic provides details about the components available for creating extensions for SCIS.

See the [Overview](#) topic of Extensibility API reference for information about each component and its methods and properties.

Extensibility API Components

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount | SuiteCommerce InStore

A component is a feature in a Commerce application that connects with certain parts of the application to accomplish a task. The Extensibility API is organized around these components. Each component has a class and various exposed methods for obtaining or modifying data, views, or template context.



Important: The Backend Extensibility API has been deprecated since the 2020.1 release of SuiteCommerce and SuiteCommerce Advanced. Instead, you should use methods available in the Frontend Extensibility API or SuiteScript 2.0 modules and methods. For more information, see the [Extensibility API Reference](#) and the [SuiteScript 2.x API Introduction](#).

SuiteCommerce, SuiteCommerce Advanced, and SuiteCommerce InStore each have their own set of components. The Extensibility API includes the following components.

- [SuiteCommerce and SuiteCommerce Advanced Components](#)
 - Cart
 - Checkout
 - CMS
 - Environment
 - Layout
 - LoginRegisterPage
 - MyAccountMenu
 - PageType
 - PageTypeBaseView
 - PDP (Product Details Page)
 - PLP (Product List Page)

- SCCollection
- SCCollectionView
- SCFormView
- SCModel
- SCView
- Search
- UserProfile
- **SuiteCommerce InStore (SCIS) Components**
 - Cart
 - SCISLayout

For more information about how to use these components when building extensions, see the help topic [Extension Tutorials](#).

SuiteCommerce and SuiteCommerce Advanced Components

The SuiteCommerce components apply to both SuiteCommerce and SuiteCommerce Advanced.

See the full [Overview](#) topic of SuiteCommerce Extensibility API for detailed information about each component.

Cart

This component interacts entirely with the LiveOrder, allowing you to add, update, and remove lines from the order, apply promotions to the order, select a shipping method on the order, associate the shipping and billing address to the order, add payment methods, and so on. This component can access objects available in all three ssp applications: shopping, myaccount and checkout.

ID	Cart
Methods	addLine() addLines() addPayment() addPromotion() clearEstimateShipping() estimateShipping() getAddresses() getBillAddress() getLatestAddition() getLines() getPromotions() getPurchaseOrderNumber() getShipAddress() getShipMethod() getShipMethods()

	getSummary() removeLine() removePromotion() removeShipping() setPurchaseOrderNumber() setShipMethod() setTransactionBodyField() submit() triggerAddToCartConfirmation() updateLine()
Events	beforeAddLine afterAddLine beforeAddPayment afterAddPayment beforeAddPromotion afterAddPromotion beforeClearEstimateShipping afterClearEstimateShipping beforeEstimateShipping afterEstimateShipping beforeRemoveLine afterRemoveLine beforeRemovePromotion afterRemovePromotion beforeSetPurchaseOrderNumber afterSetPurchaseOrderNumber beforeSetShipMethod afterSetShipMethod beforeSetTransactionBodyField afterSetTransactionBodyField beforeSubmit afterSubmit beforeUpdateLine afterUpdateLine

Checkout

This component interacts entirely with the Checkout page and the checkout steps flow. This component lets you modify the steps visually, such as adding, removing, and reordering steps, etc. This component can access objects available in the checkout application only.

ID	Checkout
Methods	getCurrentStep() setCurrentStep() getStepGroupsInfo() getStepsInfo() getCheckoutFlow() addModuleToStep() removeModuleFromStep() addStepsGroup() removeStepsGroup() addStep() removeStep()
Events	beforeAddModuleToStep afterAddModuleToStep beforeSetCurrentStep afterSetCurrentStep beforeAddStep afterAddStep beforeAddStepsGroup afterAddStepsGroup

CMS

This component is created only for the purpose of registering new Custom Content Types. If you want to add views inside of your CCT you can interact with all the rest of the components available. This component can access objects available in all three ssp applications: shopping, myaccount, and checkout.

ID	CMS
Methods	registerCustomContentType()
Events	–

Environment

This component is general to the entire SuiteCommerce Application and provides the utils method and access to Configuration values. This component does not provide interaction with any particular view. This component can access any object available in all three ssp applications: shopping, myaccount and checkout.

ID	Environment
Methods	addTracker() defineSearchMasterOptions()

	getConfig()
	isPageGenerator()
	getSiteSetting()
	getSession()
	setTranslation()
	triggerEvent()
Events	–

Layout

This component lets you modify aspects of SuiteCommerce that do not have a specific component available. For example, no components currently exist that let you add or modify views on the home page, in the header, or in the footer. Using Layout, you can access all the view methods for each view (addChildViews, addChildView, removeChildView, and so on). As more components become available in future SuiteCommerce updates, you can replace the use of Layout for the appropriate (new) component, keeping the same methods.

ID	Layout
Available Since	SuiteCommerce 2018.2
Methods	addChildView() addChildViews() addToViewContextDefinition() addToViewEventsDefinition() closeMessage() registerView() removeChildView() removeToViewContextDefinition() removeToViewEventsDefinition() setChildViewIndex() showContent() showMessage()
Events	–

LoginRegisterPage

The LoginRegisterPage component is used to listen for login and register events on the Log In / Register page in the web store. It also enables you to capture additional form data from the login and registration forms.

ID	LoginRegisterPage
Available Since	SuiteCommerce 2019.1
Methods	–

Events	beforeLogin beforeRegister afterRegister
---------------	--

MyAccountMenu

This component lets you add menu items to the menu on the My Account page. You can add group items and then add subitems to the group items. All group items and subitems have one or more permissions, which lets you restrict which users can view the menu items. Subitems point to a landing page URL that is loaded in the main content area of the page.

ID	MyAccountMenu
Available Since	SuiteCommerce 2019.1
Methods	addGroup() addGroupEntry()
Events	–

PageType

Use the PageType component when you want to register new page types in an extension. A page type record can be created by logging in to NetSuite and creating the record, or by including the page type in the extension manifest file.

ID	PageType
Available Since	SuiteCommerce 2019.1
Methods	registerPageType() registerTemplate()
Events	–

ProductDetailsPage (PDP)

This component interacts entirely with the Product Details page. This accesses objects inside the Product Details View only. This is undefined if invoked from any other part of the application.

ID	PDP
Methods	getAllMatrixChilds() getItemInfo() getPrice() getSelectedMatrixChilds setOption() setQuantity() getStockInfo()

Events	beforeOptionSelection
	afterOptionSelection
	beforeQuantityChange
	afterQuantityChange
	beforeImageChange
	afterImageChange

ProductListPage (PLP)

This component interacts entirely with the Search Page, allows navigating to the next or previous page, set the current page, change the rendering, search for a product, get the last search term, and so on. This component accesses objects inside the Facets Browse View in the shopping application only. This is undefined if invoked from any other part of the application.

ID	PLP
Methods	getPagination()
	setCurrentPage()
	getSorting()
	getAllSorting()
	setSorting()
	getDisplay()
	getAllDisplay()
	setDisplay()
	getPageSize()
	getAllPageSize()
	setPageSize()
	getFilters()
	getAllFilters()
	setFilters()
	getSearchText()
	setSearchText()
	getItemsInfo()
	getCategoryInfo()
	getURL()
Events	–

SCCollection

The SCCollection component is the SuiteCommerce implementation of Backbone.Collection. It inherits all the methods and properties of Backbone.Collection. See the [Backbone.js](#) website for more information about Backbone.Collection.

Available since SuiteCommerce 2020.1.

SCCollectionView

Use the SCCollectionView component when you want to work with a collection of elements in a grid pattern on a web store page. SCCollectionView contains the SCCollectionView abstract class, which you must override with your own child class. Properties and methods in the class let you specify how rows and columns are displayed on the page. See the [extensibility API reference](#) for detailed information.

Available since SuiteCommerce 2020.2.

SCFormView

The SCFormView component enables you to validate form field data, display error messages in a form, and save the data to the associated model. The SCFormView class is an abstract class, which you must override with your own child class.

Available since SuiteCommerce 2020.2.

SCModel

The SCModel component is the SuiteCommerce implementation of Backbone.Model. It inherits all the methods and properties of Backbone.Model. See the [Backbone.js](#) website for more information about Backbone.Model.



Important: In extensions on SuiteCommerce 2020.1 or later, you should use SCModel instead of Backbone.Model.

Available since SuiteCommerce 2020.1.

SCView

The SCView component is the SuiteCommerce implementation of Backbone.View. This component contains an abstract class, SCView, which you must override with your own child class. It provides all methods and properties of Backbone.View, as well as additional methods and properties to work with SuiteCommerce. For example, instead of using the native events property of Backbone.View, you should use the `getEvents()` method of SCView. See the [extensibility API reference](#) for detailed information.



Important: In extensions on SuiteCommerce 2020.2 or later, you should use SCView instead of Backbone.View.

Available since SuiteCommerce 2020.2.

Search

The main purpose of the Search component is to retrieve the URL of the item search API of the SuiteCommerce website. You can then use the URL to search for items or get item information. The returned URL only includes the path and parameters of the item search API for a website; it does not include the URL scheme or domain name. Here is an example of a URL returned by the `getUrl()` method of this component:

```
/api/items?
language=en&country=US&currency=USD&pricelevel=5&use_pcv=F&q=frames&fieldset=relateditems&dimensions=52x30
```

ID	Search
----	--------

Available Since	SuiteCommerce 2021.1
Methods	getUrl()
Events	–

Available since SuiteCommerce 2021.1.

UserProfile

The UserProfile component lets you retrieve information related to the user logged in to the SuiteCommerce website. You can get basic user profile information as well as the user's credit card data.

ID	UserProfile
Available Since	SuiteCommerce 2019.1
Methods	getCookieOptions() getCreditCards() getCustomerSegments() getUserProfile() setCookieOptions()
Events	–

SuiteCommerce InStore (SCIS) Components

The SCIS (SuiteCommerce InStore) components in the extensibility API let you create extensions that add, remove, or update lines in the cart. You can also show modal dialogs to present information to the user, and require the user to confirm or cancel an action.

See the full [extensibility API reference](#) for detailed information about each component.

Cart

The Cart component in SCIS lets you add, remove, and update lines in the cart. It also provides methods to perform other operations in the cart, such as applying promotions and discounts.

ID	Cart
Available Since	SuiteCommerce 2018.2
Methods	addGlobalDiscount() addLine() addLines() addLineDiscount() addPromotion() getLines() getSummary() removeLineDiscount() removePromotion()

	returnLine() setCustomer() unvoidLine() voidLine()
Events	beforeAddGlobalDiscount afterAddGlobalDiscount beforeAddLine afterAddLine beforeAddLineDiscount afterAddLineDiscount beforeRemoveLineDiscount afterRemoveLineDiscount beforeReturnLine afterReturnLine beforeSetCustomer afterSetCustomer beforeUnVoidLine afterUnVoidLine beforeVoidLine afterVoidLine

SCISLayout

The SCISLayout component gives you access to two methods, both of which show a modal dialog to the user, but with different options to dismiss the dialog.

ID	SCISLayout
Available Since	SuiteCommerce 2018.2
Methods	showConfirmationPopup() showMessagePopup()
Events	–

Instantiating API Components

❗ Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

To use an extensibility API component in your extension, you usually have to instantiate the class in the component first. To instantiate the class, pass the ID of the class to the `getComponent()` method on the container object. It is also good practice to check that the component exists before accessing its methods or properties. If the component instantiation fails, `getComponent()` returns `null`.

```
1 | var pdp = container.getComponent('PDP');
2 |
```

```

3 | if (pdp) {
4 |   var stockinfo = pdp.getStockInfo();
5 | }
6 | else {
7 |   console.log('Could not instantiate PDP.');
```

Some components in the extensibility API expose classes that do not need to be instantiated with `getComponent()`. Instead, classes in the following components are added as dependencies when defining modules such as models, views, and collections:

- SCCollection
- SCCollectionView
- SCFormView
- SCModel
- SCView

See the [extensibility API reference](#) for more information about each component.

Access the Frontend Components

To access the frontend components:

1. Access a frontend component in the `mountToApp()` function in the entry point for your extension.

By default, the entry point file is located in the JavaScript subdirectory in your extension code and uses the following naming convention:

```
1 | <Vendor.ExtensionName.ModuleName>.js
```



Important: Some component methods need to be used after the main view of the component has been rendered (after the event `afterShowContent` has been triggered). This applies to the PDP, PLP, and Checkout components.

2. Define the dependencies:

```

1 | define(
2 |   'Vendor.ExtensionName.1.0.0.Extension'
3 |   , [
4 |     'Vendor.ExtensionName.1.0.0.ExtensionName.Model'
5 |     , 'Vendor.ExtensionName.1.0.0.Extension.View'
6 |   ]
7 |   , function (
8 |     ExtensionModel
9 |     , ExtensionView
10 |   )
```



Note: In the above example, the values for `Vendor` and `ExtensionName` depend on the values you enter when you create the baseline extension. The names `ExtensionModel` and `ExtensionName` are example names for your model and view, respectively, that you define when you create the extension code.

3. Instantiate the component with the `getComponent()` method on the container object.

Since you can use the same entry point file for all three applications (MyAccount, Checkout, or Shopping), make sure that your component is not null before using it.

The following example shows how to call the PDP component and test to see that it exists before passing it to the rest of the models and views:

```

1 {
2   'use strict';
3
4   return {
5     mountToApp: function mountToApp (container) {
6
7       var pdp = container.getComponent('PDP');
8
9       if (pdp) {
10        pdp.addChildViews();
11
12        new ExtensionModel({pdp: pdp});
13      }
14    }
15  }
16 });

```

Access the Backend Component

Access `BackEndCartComponent` anywhere in the SuiteScript code, using the `Application.getComponent(component_name)` method.

Currently, only the `BackEndCartComponent` component is available to SuiteScript code.

To access the backend component

1. Define the dependencies:

```

1 define('Vendor.ExtensionName.1.0.0.Extension.File'
2 , [
3   'Application'
4   , 'underscore'
5 ]
6 , function(
7   Application
8   , -
9 ) {

```

To access the backend component, you must include `Application` as a dependency.

Note: In the above example, the values for `Vendor` and `ExtensionName` depend on the values you entered when you created the baseline extension. The name `Extension` is an example name that you define when you create the extension code.

2. Use the `getComponent()` method to access the `BackEndCartComponent` instance:

```

1 {
2   'use strict';
3
4   // Get the cart component
5   var cart_component = Application.getComponent('Cart');
6 };

```

Use SuiteScript With Your Extension

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

SuiteScript is NetSuite's proprietary scripting language that provides custom objects and methods you can use with JavaScript. The SuiteScript API provides access to NetSuite data. In terms of developing

SuiteCommerce extensions, the SuiteScript API plays an important role in backend models, where NetSuite data is created, read, updated, transformed, and validated. The SuiteScript API lets you access objects such as:

- Transaction objects like the contents of a shopping cart or an order
- Standard, non-transactional records like customer records
- Custom records

Review the following information to learn more about the SuiteScript API:

See topic	To learn
SuiteScript Overview	About the differences between SuiteScript versions, including 1.0, 2.0, and 2.1, and to access topics that explain more about what you can do with SuiteScript like using SuiteScript with NetSuite records.  Note: SuiteScript 2.1 is not currently supported for Commerce website development. SuiteScript 2.x, 2.0, and 1.0 are supported.
SuiteScript Developer Guide	How to set up your environment for SuiteScript, SuiteScript governance, best practices and more.
SuiteScript Records Guide	More about the NetSuite records available to you when using SuiteScript.
SuiteScript 2.x	More about all aspects of SuiteScript 2.0 and 2.1, including API introduction, script types, custom modules.
SuiteScript 2.x API Reference	Details about API objects and modules.

Item Search API

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce InStore

The Item Search API is a configuration driven API, which is determined by your Search Settings. For example, items returned in the API response data can be filtered by the facets you defined on the Search Index subtab when you set up Search Settings. For more information, see the help topic [Search Settings Overview](#).

The Item Search API uses a REST/JSON style of communication, and supports searching, sorting, faceting, pagination, or any combination of these as they can operate independently. Web Developers can use this API to build facet filtering and keyword search on item data.

i Note: To use the Item Search API, you must have a Commerce web store or SCIS, and you must have configured item search settings for your site.

See the following topics for more information about the Item Search API:

- [Items Available to the Item Search API](#) – This topic describes the factors that affect whether an item can be retrieved by the Item Search API.
- [The Base URL](#) – The base URL is the URL sent in the HTTP request to the Item Search API in order to retrieve a list of items. This topic describes the structure of the base URL.
- [Item Search API Input Parameters](#) – This topic describes all the input parameters that can be used in query strings to retrieve item record attributes.
- [Item Search API Output Response](#) – This topic describes all the properties included in the Item Search API output response.
- [Sample Item Search Query and Results](#) – This topic provides sample Item Search API queries and descriptions of response data.

Items Available to the Item Search API

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce InStore

The Item Search API returns data from all item records that have been indexed. For more information about the search index, see the help topic [Search Index Overview](#).

The Item Search API returns the items depending on the item availability in a given subsidiary at the time of indexing. Therefore, items are returned only if the following conditions are met:

- If the Subsidiaries feature is enabled for your account, the item should be associated with a subsidiary that is online in the website.
- The item should be associated with an income account. For more information, see the help topic [Account Information on Items](#).
- The Display in Web Site box should be selected on the item record.
- The Inactive box should not be selected on the item record.
- The item should be assigned to a Site Builder Category or a Commerce Category, or the Show Uncategorized Items box should be selected on the Web Site Setup page. For more information, see the help topic [Search Index Settings](#).
- The item should be in stock, or the selected Out of Stock Behavior option should display the item even when out of stock.
- The item type should not be Other Charge Item or Discount Item. For more information, see the help topic [Item Types](#).
- The item price should be a numeric value. For example, some item types such as Discounts can have prices in percentage (10% off instead of \$10 off).

Note: All item records flagged to display in the web site are accessible to the Item Search API. Since the Item Search API does not provide any authentication, the data retrieved by this service is exposed to public traffic on the internet.

Important: If you are using an earlier release of SuiteCommerce Advanced, you require a patch to ensure that the Item Search API response is cached in the CDN. For more information, see the help topic [Item Search API Response Data not Cached](#).

The Base URL

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce InStore

The Item Search API is typically consumed by an AJAX client. You can use a client-side JavaScript framework like jQuery to send an HTTP request to the API and get the JSON response back. The URL is accessible by any client that supports HTTP and JSON. You can retrieve results for a particular item search by sending an HTTP GET request.

The Item Search API base URL has the following format:

`http://www.mywebstore.com/api/items`

The components of the base URL are described here:

- **Custom Domain** – A unique URL is required, which must be a custom domain. For example, `www.mywebstore.com` is a custom domain.
- **Items** – This API is designed to query item records in your NetSuite account. Note that `/api/items` in the URL is part of the path and it is required. You cannot assign a value to it.
- **Parameters** – You can use parameters defined by NetSuite along with values from your account. You can use multiple parameters in an Item Search API request by separating them with an ampersand (&). Parameters are typically name-value pairs, such as `fieldset=details`. For more information, see [Item Search API Input Parameters](#).

Note: JSONP (JSON with padding) wraps the JSON response from API requests in a JavaScript variable that can be included in a script tag to enable cross-domain AJAX requests. Web developers can use `callback` as the input parameter as described in [Callback Function](#), and then create the variable name. For more information about the jQuery API, see api.jquery.com/jquery.getJSON/.

Cacheable and Personalized Endpoints

The Item Search API has two additional endpoints:

- `/cacheable/items` – responses are cached. This endpoint works in the same way as the `/items` endpoint.
- `/personalized/items` – responses are never cached. This endpoint retrieves personalized item information that is specific to a customer.

Since the 2021.2 release of SuiteCommerce and SuiteCommerce Advanced (SCA), cacheable or personalized API endpoints are automatically used depending on whether a user is logged in to your Commerce website and whether the personalized endpoint configuration option has been enabled.

Example: Personalized API endpoints are used if the user is logged in and you have enabled the personalized endpoint configuration option.

If the user is not logged in, any of the following URLs can be used as both endpoints work the same way:

- `http://www.mywebstore.com/api/items`
- `http://www.mywebstore.com/api/cacheable/items`

The URL `http://www.mywebstore.com/api/items` can be used for SCA versions 2021.1 and earlier.

Personalized API endpoints are enabled by default. However, you can verify that they are enabled by checking the SuiteCommerce Configuration.

 **Note:** To ensure that logged-in users receive personalized catalog views that are not cached for other users, you should enable personalized API endpoints.

To verify that the personalized API endpoints are enabled:

1. In NetSuite, go to Commerce > Websites > Configuration.
2. Select your website and domain and click **Configure**.
3. Go to the **Shopping Catalog** tab.
4. Check the **Enable New Search API Endpoints** box.
5. Click **Save**.

For information about the configurable property, see the help topic [Enable Personalized Search API Endpoints](#).

Item Search API Input Parameters

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce InStore

The Item Search API Input Parameters are used to construct query strings that return item record attributes in a JSON response. To create a query string for the Item Search API, you should use the input parameters with the base URL `http://www.mywebstore.com/api/items?{parameters}`. You can also use multiple parameters with the base URL by separating them with an ampersand (&).

Note: Some parameters, when used with the base URL, return the item IDs of the matching items. However, if you want to retrieve additional item details, you can use the parameters listed under the Retrieve Additional Item Details section with these parameters.

The base URL when used without any parameters retrieves the item IDs of all the items. To retrieve only the desired items and item details, you can specify the following input parameters:

Parameters	Usage
c n region	Retrieve Items by NetSuite Account Number, Commerce Site ID, Subsidiary ID
id	Retrieve Items by Item ID
url	Retrieve Items by Item URL
q	Retrieve Items by Keyword Search
commercecategoryid commercecategoryurl commercecategoryname	Retrieve Items by Commerce Categories
use_pcv pcv_entity pcv_groups	Retrieve Items from Personalized Catalog for a specific customer or one or more customer groups
fields fieldset	Retrieve Additional Item Details
matrixchilditems_fieldset correlateditems_fieldset relateditems_fieldset	Flexibility in Specifying the Field Sets of Related Items
limit offset	Pagination of Query Results
<field_name> <numeric_facet_field_ID>.ranges <numeric_facet_field_ID>.from <numeric_facet_field_ID>.to	Facet and Filter Items

Parameters	Usage
include facet.exclude	Include or Exclude Item Attributes
sort	Sort Items
pricelevel	Specify Price Level
currency	Specify Currency
language country	Specify Language and Country
callback	Callback Function
ssdebug	Retrieve Debug Information

Retrieve Items by NetSuite Account Number, Commerce Site ID, Subsidiary ID

- **c** — You can use the parameter **c** to specify the NetSuite account and retrieve all the items in the specified NetSuite account. For example, if your NetSuite account is 3925062, you can use the following query string to retrieve all the items that belongs to your NetSuite account:
`http://www.mywebstore.com/api/items?c=3925062`
- **n** — You can use the parameter **n** to specify the ID of your Commerce web store and retrieve all the items in the specified Commerce web store. For example, if the ID of your Commerce web store is 3, you can use the following query string to retrieve all the items that belongs to this Commerce web store:
`http://www.mywebstore.com/api/items?n=3`
- **region** — You can use the parameter **region** to specify the Subsidiary in a NetSuite OneWorld account and retrieve all the items in the specified Subsidiary. For example, if your Subsidiary ID is 4, you can use the following query string to retrieve all the items that belongs to this Subsidiary:
`http://www.mywebstore.com/api/items?region=4`

You can also use these parameters together to further refine the search results. For example, if have more than one Commerce web store in a NetSuite account and you would like to retrieve items belonging to a particular site, you can use the following query string:

`http://www.mywebstore.com/api/items?c=3925062&n=3`



Note: If you specify an incorrect NetSuite account number, an incorrect Commerce Site ID, an incorrect or inactive Subsidiary ID, or if your website is not reachable, no items are returned and an error message is displayed.

Retrieve Items by Item ID

You can use the parameter **id** to specify the item record ID and retrieve information for a particular item. For example, you can use the following query string to retrieve the item with Item ID 123:

`http://www.mywebstore.com/api/items?id=123`

To return multiple items in response data, list multiple item IDs separated by commas. For example, you can use the following query string to retrieve the items with Item ID 123, 456, 789:

`http://www.mywebstore.com/api/items?id=123,456,789`

Note: If the item is marked as Inactive or if the Display in Web Store box is not selected on the item record, no items are returned. If you specify an invalid item ID, an error message is displayed. In case of Matrix items, active child items are returned only if the parent item is active and the Display in Web Store box is selected.

Retrieve Items by Item URL

You can use the parameter `url` to specify the value in the URL Component field on the item record. The `url` parameter returns the Item ID of item that matches the URL component value in the request.

`http://www.mywebstore.com/api/items?url=Vok-500-Cellular-Phone`

You can specify other parameters along with the `url` parameter to retrieve other item details. For example, you can use the `fields` parameter to retrieve associated images:

`http://www.mywebstore.com/api/items?url=Vok-500-Cellular-Phone&fields=itemimages_detail`

You can also use a field set that contains image information:

`http://www.mywebstore.com/api/items?url=Vok-500-Cellular-Phone&fieldset=details`

Note: If the item is marked as Inactive or if the Display in Web Store box is not selected on the item record, no items are returned. If you specify an invalid item URL, an error message is displayed. In case of Matrix items, active child items are returned only if the parent item is active and the Display in Web Store box is selected.

Retrieve Items by Keyword Search

You can use the parameter `q` to retrieve all the items that match the specified keyword. For example, `q=jeans` searches for that keyword in the fields listed on the Search Fields subtab.

`http://www.mywebstore.com/api/items?q=jeans`

Note: Matching items are returned only if the specified keyword exists in the search fields that you selected on the Web Site Setup page. For information about selecting Search Fields, see the help topic [Select Search Fields](#).

To do a multi-term search, specify two or more terms in the URL separated by whitespace. For example, `q=denim jeans` searches for these keywords in the fields listed on the Search Fields subtab.

`http://www.mywebstore.com/api/items?q=denim jeans`

Note: Relevance is the default sort order when you use the `q` parameter to query the Item Search API. Your preferred sort order specified on the Sort Fields subtab is overridden. For information about selecting and configuring Sort Fields, see [Select and Configure Sort Fields](#).

Retrieve Items by Commerce Categories

- `commercecategoryid` — You can use the parameter `commercecategoryid` to retrieve items that belong the specified Commerce Category ID.

`http://www.mywebstore.com/api/items?commercecategoryid=12345`

- **commercecategoryurl** — You can use the parameter `commercecategoryurl` to retrieve items that belong to the specified Commerce Category URL.

`http://www.mywebstore.com/api/items?commercecategoryurl=/apparel/partywear/shoes`

- **commercecategoryname** — You can use the parameter `commercecategoryname` to retrieve items that belong to the specified Commerce Category.

`http://www.mywebstore.com/api/items?commercecategoryname=Summer Sale`

This parameter is dependent on the Commerce Category facet field being defined in the Web Site Setup record. For more information, see the help topic [Add Commerce Categories to Website Search Index](#).

Note: For a sample search query and response on Commerce Categories, see [Sample Search Query for Returning Categories](#).

Retrieve Items from Personalized Catalog for a specific customer or one or more customer groups

Important: To be able to use the `pcv_entity` and `pcv_groups` parameters, the user must have a valid NetSuite user account and be assigned a role that provides edit website or list items permissions. For more information, see the help topic [Personalized Catalog Views Roles and Permissions](#). Note that these parameters are intended for administrative interfaces only and not for building web pages that display a personalized catalog.

- **use_pcv** — For the Personalized Catalog Views feature to take effect while retrieving items, you must use the `use_pcv` parameter with the Item Search API. When the `use_pcv` parameter is set to true, the Item Search API returns tailored selection of catalog items from the user's customer groups. The Item Search API query without the `use_pcv` parameter has the same effect as `use_pcv=F`. Therefore, when the `use_pcv` parameter is set to false, the Item Search API serves items for anonymous users regardless of whether the user is logged in or recognized.

For example, a NetSuite user with sufficient permissions for PCV posts the following query:

`http://www.mywebstore.com/api/items?use_pcv=T`

The Item Search API lists all the items but only shows the items which should be visible for this user.

Note: By default, the Item Search API response data has a volatility of Short or Medium. Volatility refers to the CDN cache settings for your Commerce implementation. If the `use_pcv` parameter is set to true, the Item Search API always serves non-cacheable responses, which is evident in the Item Search API response where `volatility=unique`. The Item Search API query without the `use_pcv` parameter has the same effect as `use_pcv=F`. Therefore, if the `use_pcv` parameter is set to false, the Item Search API uses the existing caching behavior.

- **pcv_entity** — You can use the `pcv_entity` parameter to specify a single entity ID to be used as the basis for customer groups to apply query filtering. For the `pcv_entity` parameter to take effect while retrieving items, you must also specify `use_pcv=T` with the `pcv_entity` parameter.

For example, let's consider there are items on the site assigned to these four customer groups: ALL_USERS, GUEST_USERS, RECOGNIZED_USERS and VIP_USERS. Items assigned to the VIP_USERS customer group are exclusive items and are not available for customers in other customer groups. There is a customer, say CUSTOMER1, with Customer ID 111 who belongs to the VIP_USERS customer group. A NetSuite user with sufficient permissions for PCV posts the following query:

`http://www.mywebstore.com/api/items?use_pcv=T&pcv_entity=111`

The Item Search API returns all the items available for customer with customer ID 111 in the response, which includes items from ALL_USERS, RECOGNIZED_USERS and VIP_USERS customer group.

Note: If you specify an invalid entity ID (for example, a string) or an incorrect entity ID (for example, a valid number but a non-existent entity ID) with the `pcv_entity` parameter, no items are returned in the Item Search API response. If you specify more than one entity ID with the `pcv_entity` parameter, an error message is displayed.

- `pcv_groups` — You can use the `pcv_groups` parameter to specify one or more customer group IDs to apply query filtering. For the `pcv_groups` parameter to take effect while retrieving items, you must also specify `use_pcv=T` with the `pcv_groups` parameter.

For example, let's consider there are items on the site assigned to these three customer groups: ALL_USERS (Customer Group ID= 1), GUEST_USERS (Customer Group ID= 2), RECOGNIZED_USERS (Customer Group ID= 3). A NetSuite user with sufficient permissions for PCV posts the following query:

`http://www.mywebstore.com/api/items?use_pcv=T&pcv_groups=3`

The Item Search API returns all the items assigned to the group RECOGNIZED_USERS.

Note: If you specify an invalid group ID (for example, a string) or an incorrect group ID (for example, a valid number but a non-existent group ID) with the `pcv_groups` parameter, no items are returned in the Item Search API response. If you specify two group IDs with the `pcv_groups` parameter with one existing group ID and one non-existent group ID, only the items for the existing group are returned in Item Search API response.

Retrieve Additional Item Details

If you do not specify any input parameters with the Item Search API base URL, only the Item IDs of the items are returned. You can use the following parameters to retrieve additional item details:

- `fields` — You can use the parameter `fields` to retrieve additional item record fields in the API response. Your request can include a valid field ID, or multiple field IDs separated by commas to return multiple fields.

`http://www.mywebstore.com/api/items?fields=custitem1,upccode`

- `fieldset` — You can use the parameter `fieldset` to retrieve all the item fields associated with a field set.

`http://www.mywebstore.com/api/items?fieldset=details`

Note: When the Commerce Category field is defined in the Web Site Setup record field sets, the Commerce Category details for a given item are returned.

Flexibility in Specifying the Field Sets of Related Items

- `matrixchilditems_fieldset` — You can use the parameter `matrixchilditems_fieldset` to specify the field set that overrides the reserved `matrixchilditems` field set. The `matrixchilditems` field set is used by the `matrixchilditems_detail` derived field by default to retrieve the item fields of all the matrix child items. In the following example, the default `matrixchilditems` field set is overridden with a field set named `mini` using the Item Search API parameter `matrixchilditems_fieldset`.

`http://www.mywebstore.com/api/items?fieldset=search&matrixchilditems_fieldset=mini`

- `correlateditems_fieldset` — You can use the parameter `correlateditems_fieldset` to specify the field set that overrides the reserved `correlateditems` field set. The `correlateditems` field set is used by the `correlateditems_detail` derived field by default to retrieve the item fields of all the correlated items. In the following example, the default `correlateditems` field set is overridden with a field set named `mini` using the Item Search API parameter `correlateditems_fieldset`.

`http://www.mywebstore.com/api/items?fieldset=search&relateditems_fieldset=mini`

- `relateditems_fieldset` — You can use the parameter `relateditems_fieldset` to specify the field set that overrides the reserved `relateditems` field set. The `relateditems` field set is used by the `relateditems_detail` derived field by default to retrieve the item fields of all the related items. In the following example, the default `relateditems` field set is overridden with a field set named `mini` using the Item Search API parameter `relateditems_fieldset`.

`http://www.mywebstore.com/api/items?fieldset=search&relateditems_fieldset=mini`

Pagination of Query Results

You can use the parameter `offset` in combination with another parameter `limit` to paginate Item Search API query results. The `offset` parameter specifies the offset from the first result and the `limit` parameter specifies the number of items that can be displayed on each page. For example, you can retrieve query results from the eleventh item to the fifteenth item using the following query string:

`http://www.mywebstore.com/api/items?offset=10&limit=5`

You can click next at the bottom of the query result to increment the `offset` parameter by the value of `limit`. Note that the value of `limit` remains unchanged. In the above example, when you click next, the value of `offset` changes to 15 and the next five items are displayed. Also, you can click prev at the bottom of the query result to decrease the value of the `offset` parameter by the value of `limit`. In the above example, when you click prev, the value of `offset` changes to 5 and the previous five items are displayed.

Note: The Item Search API is not designed for deep pagination. Therefore, using `offset` and `limit` to page through sorted results works well if you only need to access the first few pages of the search results.

The default value of the `offset` parameter is 0 and that of the `limit` parameter is 50. Therefore, when you do not specify `offset` and `limit`, the first 50 items are displayed in the query results.

Facet and Filter Items

- `<facet_field_name>` — You can specify the facet field name and facet values to retrieve the item IDs of the items associated with the specified facets and facet values. For example, you can filter items for the specified facets and facet values:

`http://www.mywebstore.com/api/items?color=red,blue&gender=women`

If you are unsure about the facet value or if special characters are included in the facet value you want to specify in the query, you can call the Search API requesting all facets. For example, you can request all facets using the following query string:

`http://www.mywebstore.com/api/items?pricelevel=5&include=facets`

You can then use the appropriate value that is contained in the response to construct your API query.

Note: If you create a custom list intended for use as a facet, do not include a comma in any of the list values. In an API request, the comma is interpreted as a separator between multiple options.

- `<numeric_facet_field_ID>.ranges` — You can use the parameter `<numeric_facet_field_ID>.ranges` to define range buckets for numeric facets fields such as price. Use the suffix `ranges` on the field ID to perform the search. For example, you can retrieve an array of range buckets with the number of items in each bucket using the following query string:

`http://www.mywebstore.com/api/items?pricelevel5.ranges=1-10,10-20,20-*`

The range buckets in this example are 1 to 10, 10 to 20, and 20 or more.

The range buckets used to compute range faceting are inclusive of their lower bounds and exclusive of their upper bounds. In the above example, 10 is considered in the range bucket 10 to 20 and not in the range bucket 0 to 10.

Note: Facet ranges do not filter, but instead show additional data in the facet section of the JSON response pertaining to how many items are in each range.

- `<numeric_facet_field_ID>.from<numeric_facet_field_ID>.to` — To specify a range of values to search for on item records, you can use the parameters `<numeric_facet_field_ID>.from` and `<numeric_facet_field_ID>.to`. Use the suffix `from` and `to` on the field ID to perform the search. Both `from` and `to` are required, and excluding one results in an error. Also, you can only use the field ID of the facet field in facet range filters, and not the URL component of the facet.

For example, you can retrieve all the items with a price between 10 and 20 (both inclusive) using the following query string:

`http://www.mywebstore.com/api/items?pricelevel5.from=10&pricelevel5.to=20`

Note: When the **Prices Include Tax** preference is set, tax is included in the price of each item. However, the Item Search API does not support the Tax Inclusive Pricing feature. Therefore, tax exclusive prices are used when you filter the items using any price-related facet.

Include or Exclude Item Attributes

- `include` — You can use the parameter `include` to specify an item attribute in the API request and retrieve additional data in the JSON response. For example, the following sample query includes facet fields in the JSON response:

`http://www.mywebstore.com/api/items?include=facets`

- `facet.exclude` — You can use the parameter `facet.exclude` to exclude fields from faceting in Item Search API calls. Excluding facet fields that are not essential can help speed up the API calls. The `facet.exclude` parameter accepts comma delimited facet field names.

In the following example, the API call specifies a facet value for filtering (`custitem_categories=Kitchen-Utensils`), while excluding the field from faceting (`facet.exclude=custitem_categories`).

`http://www.mywebstore.com/api/items?`

`include=facets&fieldset=search&language=en&country=US¤cy=USD&pricelevel=5&c=1234567&n=2&custitem_Utensils&s`

Sort Items

You can use the parameter `sort` to define the sort order of search results on your website. By using the field ID you want to sort by in the API request, you can retrieve item record data sorted by the field ID included in the API request. You can find the field ID on the Sort Fields subtab on the Web Site Setup page. An Item Search API request overrides your settings on the Search Fields tab. Supported sort orders are ascending (`asc`) and descending (`desc`).

For example, you can sort the items by their store display name using the following query string:

`http://www.mywebstore.com/api/items?sort=storedisplayname:asc`

Items are sorted by category when the value of `commercecategory` and either `commercecategoryid` or `commercecategoryurl` are specified. This is dependent on the Commerce Category sort field being defined in the Web Site Setup record.

For more information about setting up web site preferences for sorting in NetSuite, see the help topic [Select and Configure Sort Fields](#).

Note: If you are sorting by a field which is empty for a particular item, the item is excluded from sorting and is displayed at the end of the list.

Specify Price Level

You can use the parameter `pricelevel` to specify the price level ID for items. The price level ID can be different depending on how price levels are configured in your NetSuite account. Typically, the default online price in NetSuite is `pricelevel=5`.

`http://www.mywebstore.com/api/items?pricelevel=4&include=facets`

Note: If you have set up multiple price levels for items, ensure that you add all the price level fields as sort fields. For more information, see the help topic [Select and Configure Sort Fields](#). Also, if Alternate Price is not defined for an item but is set to be displayed in a field set, then the value of Alternate Price defaults to the Online Price. If Online Price is also not defined for the item, then the value of Alternate Price defaults to the Base Price.

When a customer with a custom price level is logged in on the web store, the price level assigned to that customer is retrieved instead of the default online price level. For example, the default online price is `pricelevel=5`, but the price level assigned to a certain customer may be `pricelevel=4`. When this customer is browsing in the web store, the customer's assigned price, `pricelevel=4` is sent in the search API call. Consequently, the custom price (`pricelevel=4`) is displayed to this customer for all the products in the web store.

Note: The `pricelevel` parameter is only usable when a user with permissions to see the requested `pricelevel` is logged in. In all other cases, the `pricelevel` displayed is in accordance with the default setting in the Web Site Set Up record.

Specify Currency

Prices are usually displayed in the default currency that you have set up for your website. In an account that uses the Multiple Currencies feature, you can use the parameter `currency` with the ISO Country Code to specify the currency format in which you want to display prices for the items.

`http://www.mywebstore.com/api/items?
fields=onlinecustomerprice_formatted,pricelevel5¤cy=USD`

If you are using the Multiple Currencies feature, ensure that you select the correct default locale on the currency record to avoid any errors related to currency mismatch. For example, select New Zealand (English) as the default locale for New Zealand Dollar. For more information, see the help topic [Creating Currency Records](#).

Note: If you specify a currency that is not associated with your website or if the Online box is not checked for that currency on the Web Site Setup record, an error message is returned.

Specify Language and Country

A shopper on your website can use their local language to search and browse products on your website. The translated text that shoppers see in product views is based on the translated text you set up on item records. The JSON response data includes item attributes and field set fields in the languages you have set up in your NetSuite account. You must add the corresponding fields to a field set and also check that the field set is included in the Reference ShopFlow SuiteApp. By default, items are returned with fields containing the default language. You can use the language and country parameter to specify the translated text you set up in NetSuite.

In the following example, the search results are displayed in Spanish:

`http://www.mywebstore.com/api/items?fieldset=search&country=ES&language=es`



Note: You must always use the language and country parameters together. The country code that you specify should be in uppercase and the language code should be in lowercase.

All ISO country and language codes are valid if the language is set up for use on your website. To affect the language displayed in a product view, the site template must be localized. For more information about localizing site templates and using multiple languages, see the help topic [Localization](#).

Callback Function

You can use the parameter `callback` to define the JavaScript callback function to wrap the JSON response.

For example, you can create a variable name `mycallback` by using the `callback` parameter as shown in the following sample query:

`http://www.mywebstore.com/api/items?callback=mycallback`

Retrieve Debug Information

You can use the parameter `ssdebug` and set its value to `true` to retrieve the debug information in the JSON response.

`http://www.mywebstore.com/api/items?ssdebug=true`

Item Search API Output Response

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce InStore

The response data includes the following properties:

- **total** – This is the number of items that match the search query, but without taking into account the stock level variance since the last search index update. Consequently, the number of items displayed on the page might be less than the total displayed.

For example, if your search query matches only nine items out of the 1000 items in your catalogue, the value of total displayed is nine. If three out of these nine matching items go out of stock since the last search index update, only six items are displayed even though the value of total displayed is nine.

Note: If you use the `url` parameter to request an item by specifying the `url` component value, `total` is not displayed in the Item Search API output response.

- **items** – This is an array of items that includes the fields specified using the `fields` or `fieldset` parameter. Note that the following output for the `items` property shows fields that are included in the search field set, which was specified in the sample query:

```

1  items: [
2    {
3      ispurchasable: false,
4      custitem_ns_pr_attributes_rating: "",
5      showoutofstockmessage: true,
6      custitem38: false,
7      custitem33: "&nbsp;";
8      itemid: "OL5299",
9      onspecial: false,
10     onlinecustomerprice: 19.99,
11     pricelevel5: 19.99,
12     outofstockbehavior: "Disallow back orders but display out-of-stock message",
13     storedescription2: "Features<BR>-Cool.Q™ ZERO provides an immediate and ongoing cooling sensation<BR>-Mesh
14     sides for ventilation<BR>-Lightweight foam brim folds up and packs down small for shoving in a pack<BR>-
15     Drawcord attachment under the chin",
16     storedisplayname2: "Chiller Wide Brim Hat",
17     custitem_onsale: false,
18     internalid: 6548
19   }
20 ],

```

- **facets** – This shows the facets associated with items returned in the response data. Use the `include` parameter to show all of facets you defined in on the Web Site Setup page. Note the following sample from the output response shows IDs for two facets, `Ideal For` and `Collection`. Each facet has two facet values.

```

1  facets: [
2    {
3      id: "Ideal For",
4      values: [
5        {
6          url: "Men",
7          label: "Men"
8        },
9        {
10         url: "Women",
11         label: "Women"
12       }
13     ]
14   },
15   {
16

```

```

17     id: "Collection",
18     values: [
19       {
20         url: "Spring",
21         label: "Spring"
22       },
23       {
24         url: "Fall",
25         label: "Fall"
26       }
27     ]
28   },
29   {

```

For more information about defining facets in NetSuite, see the help topic [Select and Configure Facet Fields](#).

- links** – These links support pagination and are automatically returned in response data. You can also build the URL for the next set of results using the limit and offset parameters returned by the API. You can click next href in the query results to increment the offset parameter by the value of limit and view the next set of results. You can also click prev href in the query results to decrease the value of the offset parameter by the value of limit. Note that the value of limit remains unchanged.

```

1  links:  [
2    {
3      rel: "next",
4      href: "http://www.mywebstore.com/api/items?pricelevel=5&offset=10&limit=5&fieldset=search&include=facets"
5    },
6  ],

```

- corrections** – This property displays the past and present values for facet URL components in the Item Search API request. If you have not entered any aliases in NetSuite for facets or facet values you used in the past, then the corrections property does not return any data.

For example, in the past you may have used `custitem_category` as a facet. At that time, the facet value was Hockey Player. Today, you are using Equipment Category as the facet, and the current facet value is Player. When you modify the facet URL values, you can add the original values to the Alias list in NetSuite. If you query the Item Search API using the original facet names and values, then corrections are returned in the response data. For more information about entering URL component aliases, see the help topic [URL Components for Facets](#).

Since the following API request includes the original facet and facet value, the output response to the following query string includes corrections:

```
http://www.mywebstore.com/api/items?
pricelevel=5&fieldset=search&include=facets&custitem_category=HockeyPlayer
```

Note that both past and current facet URL components are included in the response data:

```

1  corrections:
2    [
3      {
4        type: "facet",
5        id: "custitem_category",
6        usedAlias: "custitem_category",
7        url: "EquipmentCategory"
8      },
9      {
10       type: "facet-value",
11       facet: "custitem_category",
12       usedAlias: "HockeyPlayer",
13       url: "Player"
14     }
15   ],

```

- locale** – This is an object that represents the locale with respect to internationalization. You can troubleshoot errors related to currency mismatch by verifying the details in the locale section. When

you are using the Multiple Currencies feature, you should always select the correct default locale on the currency record. For example, select New Zealand (English) as the default locale for New Zealand Dollar.

```
1 | locale: {
2 |     country: "US",
3 |     language: "en",
4 |     currency: "USD",
5 |     region: 17
6 | },
```

For more information, see the help topic [Localization](#).

- **volatility** – This refers to the CDN cache settings for your Commerce implementation. CDN caching can be set for a shorter period of time or a longer period of time. The length of time that a certain type of content is cached on CDN depends on the value of TTL (Time To Live) in CDN, which could be one of the following:
 - Unique — This asset is not cached.
 - Short — This asset may change frequently, so cache it for five minutes.
 - Medium — This asset may or may not change frequently, so cache it for two hours.
 - Long — This asset is not expected to change frequently, so cache it for seven days.

By default, the Item Search API response data has a volatility of **Short**. CDN caching can be set for a shorter period of time or a longer period of time as described in [Configure CDN Caching](#). However, changing the CDN cache setting in your application can have a significant performance impact.

 **Note:** If you exclude the `pricelevel` parameter from the Item Search API URL, the volatility is automatically set to **Unique**.

In pre-Elbrus releases of SuiteCommerce Advanced, the Item Search API response data is not cached in the CDN by default. To enable caching of the Item Search API response, you must customize your implementation to include the `pricelevel` input parameter as described in [Item Search API Response Data not Cached](#).

- **code** – This is the HTTP status code. All responses should have a status code or a message that allows the client to continue processing the results or handle the error if necessary. If the request is successful, a 200 status code indicates that the request was received, understood, and has been processed.
- **warnings** – If you have specified a parameter in the query that the API does not recognize, the parameter is ignored and a warning message is displayed in the API response. In the following Item Search API response, a warning message is displayed indicating that the non-existent parameter `foo` has been ignored.

```
1 | "warnings": {
2 |     "processeduri": "http://www.mywebstore.com/api/items?fieldset=search&foo=&include=facets"
3 | },
4 |     "ignoredparams": [
5 |         "foo"
6 |     ]
7 | }
```

Sample Item Search Query and Results

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce InStore

This section provides sample Item Search API queries and their response data.

- [Sample Search Query for Returning Item Quantity](#)
- [Sample Search Query for Returning Categories](#)
- [Sample Search Query for Returning Items Available for Store Pickup](#)

Sample Search Query for Returning Item Quantity

If you use the Multi-Location Inventory feature, you can access item inventory per location using fields specifically designed to return item location details. You can use these fields to retrieve the Item Quantity details:

- Quantity Available (quantityavailable_detail)
- Quantity Backordered (quantitybackordered_detail)
- Quantity Committed (quantitycommitted_detail)
- Quantity On Hand (quantityonhand_detail)
- Quantity On Order (quantityonorder_detail)

By using these fields with the Item Search API, you can retrieve the item inventory data. You can also create a field set that includes these fields, and then use that field set in a query to the Item Search API. For information about creating field sets, see [Define Field Sets](#).



Important: When the Sales Channel Allocation feature is enabled, the item quantity details retrieved using the Quantity Available (quantityavailable_detail) field with the Item Search API might not match the value displayed in the Quantity Available field on the item record.

This behavior occurs because the Quantity Available (quantityavailable_detail) field is returning the sum of Quantity Available and Quantity Allocated to Order Reservation instead of just Quantity Available.

The following example shows a query string that returns data from the Quantity Backordered field on the item record:

`http://www.mywebstore.com/api/items?pricelevel=5&fieldset=myfieldsetwithqtybackordered`

The item quantity information is written to the JSON response, which includes internal ID of the location and quantity back ordered for each item:

```

1 items: [
2   {
3     qtybackordered_detail:{
4       qtybackordered:10
5       locations:[
6         {
7           internalid:1,
8           qtybackordered:6
9         },
10        {
11          internalid:2,
12          qtybackordered:4
13        }
14      ]
15    }
16  ]

```

```

14         ]
15     }
16 }
17 ]

```

Sample Search Query for Returning Categories

The following query returns items where the comcat fieldset is configured with a commerce category ID 3. Facet information is also returned in the response. For each item returned, category details are returned including:

- The URL fragments defined for that category.
- The commerce category name used in the facet.



Note: To successfully run this query, the Commerce Category facet field and field set must be configured in the Web Site Set Up record. For more details on working with Categories, see the help topic [Commerce Categories](#).

http://www.mywebstore_suite.com/api/items?fieldset=comcat&include=facets&commercecategoryid=3

```

1  {
2  "total": 4,
3  "items": [
4      {
5          "internalid": 388,
6          "commercecategory": {
7              "primarypath": [
8                  {
9                      "urls": [
10                         "/apparel/partywear/shoes"
11                     ],
12                     "name": "Shoes",
13                     "id": 3
14                 },
15                 {
16                     "urls": [
17                         "/apparel/partywear"
18                     ],
19                     "name": "Partywear",
20                     "id": 2
21                 },
22                 {
23                     "urls": [
24                         "/apparel"
25                     ],
26                     "name": "Apparel",
27                     "id": 1
28                 }
29             ],
30             "categories": [
31                 {
32                     "urls": [
33                         "/apparel/partywear/shoes"
34                     ],
35                     "name": "Shoes",
36                     "id": 3
37                 }
38             ]
39         },
40         "storedisplayname2": "Shoes - Reebok 9K"
41     },
42     ...
43 ],
44 "facets": [
45     {

```

```

46     "id": "commercecategoryname",
47     "values": [
48       {
49         "url": "Shoes"
50       }
51     ],
52   },
53   ...
54 ],
55   "links": [],
56   "corrections": [],
57   "locale": {
58     "country": "US",
59     "language": "en",
60     "currency": "USD",
61     "region": 1
62   },
63   "volatility": "unique",
64   "code": 200,
65   "warnings": {}
66 }

```

Sample Search Query for Returning Items Available for Store Pickup

When you use the `isstorepickupallowed` field with the Item Search API and set it to true, the Item Search API returns the item IDs of all the matching items for which store pickup is enabled. When this field is set to false, the Item Search API returns the item IDs of all the matching items for which store pickup is not enabled.

If you enable the Multi-Location Inventory feature, you can access the item quantity per location. If you use the `quantityavailableforstorepickup_detail` field with the Item Search API, you can also access the item quantity available for store pickup per location.

The following example shows a query string that returns data from the Quantity Available and Quantity Available For Store Pickup fields on the item record. Alternatively, you can create a field set that includes Quantity Available and Quantity Available For Store Pickup fields, and then use that field set in a query to the Item Search API.

`http://www.mywebstore.com/api/items?`

`isstorepickupallowed=true&fields=quantityavailable_detail,quantityavailableforstorepickup_detail`

Note: To successfully run this query, the Store Pickup Allowed facet field must be configured in the Web Site Setup record. You must also define the Available For Store Pickup (Detail) field in the Web Site Setup record field sets. For more information, see [Accessing Item Quantity Available for Store Pickup Per Location](#).

For each item returned in the JSON response, the data includes the `internalid` of the item, total item quantity available, item quantity available by location, and item quantity available for store pickup by location:

```

1  "items": [
2    {
3      "internalid": 388,
4      "quantityavailable_detail": {
5        "quantityavailable": 90.0,
6        "locations": [
7          {
8            "internalid": 2,
9            "quantityavailable": 39.0
10         }

```

```

11         {
12             "internalid":3,
13             "quantityavailable":51.0
14         }
15     ],
16 },
17     "quantityavailableforstorepickup_detail":{
18         "locations":[
19             {
20                 "internalid":2,
21                 "qtyavailableforstorepickup":39.0
22             },
23             {
24                 "internalid":3,
25                 "qtyavailableforstorepickup":51.0
26             }
27         ]
28     },
29 },
30 {
31     "internalid":387,
32     "quantityavailable_detail":{
33         "quantityavailable":150.0,
34         "locations":[
35             {
36                 "internalid":2,
37                 "quantityavailable":150.0
38             }
39         ]
40     },
41     "quantityavailableforstorepickup_detail":{
42         "locations":[
43             {
44                 "internalid":2,
45                 "qtyavailableforstorepickup":150.0
46             }
47         ]
48     },
49 },
50 {
51     "internalid":107,
52     "quantityavailable_detail":{
53         "quantityavailable":4.0,
54         "locations":[
55             {
56                 "internalid":2,
57                 "quantityavailable":3.0
58             },
59             {
60                 "internalid":3,
61                 "quantityavailable":1.0
62             }
63         ]
64     },
65     "quantityavailableforstorepickup_detail":{
66         "locations":[
67             {
68                 "internalid":2,
69                 "qtyavailableforstorepickup":3.0
70             }
71         ]
72     },
73 },
74 ],

```

In the above JSON response, we can see the following:

- The first item with internal ID 388 is available at two locations, and both locations allow store pickup for the quantity available at the respective locations.
- The second item with internal ID 387 is only available at one location, and that location allows store pickup for the quantity available.

- The third item with internal ID 107 is available at two locations, but only one location allows store pickup for the quantity available at that location.

Accessing Item Quantity Available for Store Pickup Per Location

To access the quantity available for Store Pickup per location using the Item Search API, you need to create a field set that includes fields listed under [Item Fields Related to Store Pickup](#).

To create a field set that includes information about item quantity available for store pickup:

1. Go to Commerce > Websites > Website List.
2. Click the **Field Sets** subtab.
 1. Enter a **Name** for the field set. For example, enter **Store Pickup**.
 2. Enter a **Field Set ID** for the field set. For example, enter **storepickup_details**.
 3. Select **Item** for **Record Type**.
 4. (Optional) Enter a description for the field set.
 5. In the Field Set popup window, select the store pickup related field names listed in the subsequent table.
 6. Click **Add**.
3. Click **Save**.
4. Use the field set in a query to the Item Search API. For example,
`http://www.mywebstore.com/api/items?fieldset=storepickup_details`

The store pickup related information is returned in the JSON response.

Item Fields Related to Store Pickup

The following table shows the item record fields that correspond with fields exposed to field sets:

Field Label on the Locations Subtab of the Item Record	Field Set Field Name	Field ID
Allow Store Pickup	Store Pickup Allowed	isstorepickupallowed
Quantity Available For Store Pickup	Available For Store Pickup (Detail)	quantityavailableforstorepickup_detail

Commerce API

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

The Commerce API exposes a set of shopping objects and functions, that you can use to:

- Develop web store customizations.
- Create SSP application scripts, along with the full range of existing server-side SuiteScript. For details about the SuiteScript API, see the help topic [SuiteScript 1.0 Documentation](#).

See the following topics for more information about the Commerce API:

- [Shopping Objects](#) – The Commerce API provides several shopping objects whose associated methods enable you to control all aspects of web store behavior. These topics describe all the shopping objects and their associated methods.
- [JSON Object Fields](#) – These topics describe the structure of all JSON objects that are available for use with the Commerce API shopping methods.
- [SSP Application Governance](#) – The governance model for SSP applications limits the number of usage units that can be executed by an SSP application script. This topic lists the number of usage units consumed by Commerce API shopping methods so that you can use them to create SSP applications that meet the requirements of the governance model.

Shopping Objects

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

The Commerce API includes only one object that is globally accessible in the SuiteScript context, the `webContainer` object.

The following table lists the objects that are accessible after you have gotten a `webContainer`. The table also lists the calls you can use to get a reference to each object. You need a reference to a shopping object to call its methods.

Object	Description	Call to get Object	For available methods, see:
<code>shoppingSession</code>	Tracks session context.	<code>nlapiGetWebContainer().getShoppingSession()</code>	ShoppingSession Methods
<code>customer</code>	Holds data for the logged in customer.	<code>nlapiGetWebContainer().getShoppingSession().getCustomer()</code>	Customer Methods
<code>order</code>	Holds shopping cart data and methods for placing the order.	<code>nlapiGetWebContainer().getShoppingSession().getOrder()</code>	Order Methods
<code>pageGenerator</code>	Used to generate web pages.	<code>nlapiGetWebContainer().getPageGenerator()</code>	PageGenerator Methods
<code>standardTagLibrary</code>	Implements web site tags.	<code>nlapiGetWebContainer().getStandardTagLibrary()</code>	StandardTagLibrary Methods

Note the following:

- These customer and order objects are different from NetSuite customer and order records, because they hold information within the given shopping session only.
- To access a customer or an order, in addition to getting a `webContainer`, you also need to get a `shoppingSession`.
- You can only use Commerce API shopping methods against Commerce API objects. You cannot use shopping methods against NetSuite records.
- Shopping object methods support passing of primitive strings and JSON objects. See [Input Parameters/Return Values for Shopping Object Methods](#).
- The Commerce API includes JSON objects to support shopping object methods. For details about supported fields for JSON objects, see [JSON Object Fields](#).
- For information about governance for shopping object methods, see [SSP Application Governance](#).

Input Parameters/Return Values for Shopping Object Methods

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Set Functions

Either a primitive string or a JSON object can be passed in to **set** methods, depending on their implementation. A primitive string would be used primarily to hold an object key, while a JSON object would provide a flat representation of an object's name/value pairs.

Get Functions

An optional array can be passed in as a filter to **get** methods. If field names are passed in to a **get** method, only the name/value pairs for the passed field names are returned. If no field names are passed, all name/value pairs for the expected object are returned. If no value is available for a field, null is returned for that field.

ShoppingSession Methods

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

The shoppingSession object tracks session context and can call the following methods:

- [constructDomainBridgingUrl\(link\)](#)
- [doChangePassword\(Params, newPassword\)](#)
- [getAbsoluteUrl\(domaintype,path\)](#) (Deprecated)
- [getAbsoluteUrl2\(domaintype,path\)](#)
- [getAllCustomerSegmentTypes\(\)](#)
- [getAllCustomerSegments\(\)](#)
- [getCampaignID\(\)](#)
- [getCorrelatedItems\(item\)](#)
- [getCountries\(\)](#)
- [getCountryTaxPreferences\(\)](#)
- [getCustomer\(\)](#)
- [getEffectiveShoppingDomain\(\)](#)
- [getInformationItemFieldValues\(infoItems\)](#)
- [getItemFieldValues\(items\)](#)
- [getPartnerID\(\)](#)
- [getPasswordHint\(customerEmail\)](#) (Deprecated)
- [getPaymentMethod\(paymentMethodId, fields\)](#)
- [getPaymentMethods\(fields\)](#)
- [getRedirectURL\(\)](#)
- [getRelatedItems\(item\)](#)
- [getRelatedCorrelatedItems\(\)](#)
- [getShipToCountries\(\)](#)
- [getShopperCurrency\(\)](#)
- [getShopperLanguageLocale\(\)](#)
- [getShopperPreferences\(\)](#)
- [getShopperSubsidiary\(\)](#)
- [getSiteCategoryContents\(siteCategory, recursive\)](#) (Deprecated)
- [getSiteCategoryFieldValues\(siteCategories\)](#)
- [getSiteCategoryTree\(siteCategory\)](#)
- [getSiteSettings\(fields\)](#)

- `getSSPApplication()`
- `getStates(countries)`
- `isChangePasswordRequest()`
- `isLoggedIn()` (Deprecated)
- `isLoggedIn2()`
- `isLoggedIn3()`
- `isRecognized()`
- `login(customer)`
- `logout()`
- `proceedToCheckout(checkoutSettings)`
- `registerCustomer(customer)`
- `registerGuest(guest)`
- `sendPasswordRetrievalEmail(customeremail)` (Deprecated)
- `sendPasswordRetrievalEmail2(customeremail)`
- `removeItemOptions(itemlineid)`
- `setShopperCurrency(currencyid)`
- `setShopperLanguageLocale(localeid)`
- `setShopperPreferences(prefs)`
- `setShopperSubsidiary(subsidiary)`
- `verifyEmailChange(key)`



Note: If the method includes item as a parameter, enter the internal ID as a string, not an integer.

constructDomainBridgingUrl(link)

Converts cross-domain link to domain bridging URL containing encrypted parameters. This method provides a secure way to transfer shopper data between different domains.



Note: Do not change or extract data from the domain bridging URL as the format of the URL may change. Only use the URL in the browser to retrieve and apply the shopper session data from the original domain.

Parameters

- `link` [required] {String} – Absolute or relative URL of link targeting a different shopping or checkout domain from the one the shopper is currently on

Returns

- `link` {String} – Domain bridging URL; the original URL is returned if bridging is either not possible or not necessary
- `null` – Returned if the input parameter is invalid

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

doChangePassword(Params, newPassword)

Changes the password for the customer identified by email. Returns true if the password is changed correctly.

Throws the following errors:

- `ERR_WS_INVALID_LINK` – if parameters contain a token that is expired or incomplete. Note that it is not possible to determine if the link is expired or incomplete. The same error is thrown in either case.
- `ERR_WS_INVALID_PASSWORD` – if the password is empty or exceeds the maximum allowed length of 255 characters.
- `ERR_WS_WEAK_PASSWORD` – if the password at least 6 characters.

Parameters

- `Params` – pass parameters from original request
- `newPassword` – {String} value for new password to be set

Returns

Boolean

Supported Domains

Checkout

Login Required?

no

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getAbsoluteUrl(domainType,path)



Important: In certain scenarios, the `getAbsoluteUrl(domainType,path)` method returns a relative URL instead of an absolute URL. For example, the `getAbsoluteUrl(domainType,path)` method returns a relative URL when the target domain is same as the current domain.

Therefore, the `getAbsoluteUrl(domainType,path)` method has been removed as of NetSuite Release 2017.1. Use the [getAbsoluteUrl2\(domainType,path\)](#) method instead.

Gets absolute URL for given path and domain type.

Parameters

- `domainType` [optional] {String} - value should be either "shopping" or "checkout"
- `path` [required] {String}

Returns

String

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getAbsoluteUrl2(domainType,path)

Gets absolute URL for given path and domain type.

 **Note:** This method should be used instead of the previous [getAbsoluteUrl\(domainType,path\)](#) method.

Parameters

- domainType [optional] {String} - value should be either "shopping" or "checkout"
- path [required] {String}

Returns

String

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getAllCustomerSegmentTypes()

Gets all customer segment types available in the account, that is, all defined customer groups as well as the default groups of All Users, Anonymous Users, and Recognized and Logged In Users.

Parameters

No parameters to set.

Returns

Array of objects with fields:

- id {string}
- name {string}

Supported Domains

All

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getAllCustomerSegments()

Gets all customer segments that are in use in the Customer Segment Manager. For more information, see the help topic [Managing which Customer Segments can Access Item Segments](#).

Parameters

No parameters to set.

Returns

Array of objects with fields:

- id {number}
- name {string}
- customerSegmentType {string}

Supported Domains

All

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getCampaignID()

Gets the campaign ID of the leadsource parameter from the current session.

Parameters

No parameters to set.

Returns

campaignid {String}

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getCorrelatedItems(item)

Gets array of correlated items for given item.



Note: Complete details are returned for a correlated item only if the item is:

- available in the store from which the API was called
- available in personalized catalog view for the customer (if Personalized Catalog Views is enabled - for more information, see the help topic [Personalized Catalog Views](#))

Parameters

- `item` [required]{object with values for fields}
 - `internalid` [required]
 - `itemtype` [required]
 - `itemfields` [optional] {array of item field names to query}



Note: For better performance limit the number of queried fields by specifying only fields you need in the `itemfields` array.

Returns

Array of objects of type [item](#)

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getCountries()

Gets all countries

Parameters

No parameters to set.

Returns

Array of objects with fields:

- `name` {string}
- `code` {string}
- `isziprequired` {boolean}

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getCustomer()

Gets the Customer details from the current session.

Parameters

No parameters to set.

Returns

Returns the customer shopping object. For more information, see [Customer Methods](#).



Note: The customer object holds information within the given shopping session only and is different from the NetSuite customer record.

Supported Domains

Checkout, Shopping

Login Required?

Yes

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getCountryTaxPreferences()

Returns a JSON object with tax preferences related a shopper's country. The values returned are based on the default shopper subsidiary derived from the site or the country selected by the shopper.

Parameters

No parameters to set.

Returns

- `vatregistration {boolean}` — When set to T, the VAT registration field can be displayed during checkout and the `vatregistration` can be set on the customer object.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getEffectiveShoppingDomain()

Returns name of shopping domain associated with current session.

Parameters

No parameters to set.

Returns

domainname {String} – The shopping domain related to the current session. For example: shopping.corp.netsuite.com.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getInformationItemFieldValues(infoItems)

Gets attribute values for given information items.

Parameters

- infoItems [required] {Array}

Each object in array has values for field:

- internalid [required]

Returns

Array of objects of type [item](#).



Note: Depending on item fields configured, the array of objects returned might differ.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getItemFieldValues(items)

Gets attribute values for given items.



Note: Complete details are returned for an item only if the item is:

- available in the store from which the API was called
- available in personalized catalog view for the customer (if Personalized Catalog Views is enabled - for more information, see the help topic [Personalized Catalog Views](#))

Parameters

- `items` [required] {Array} Each object in array has values for fields:
 - `internalid` [required]
 - `itemtype` [required]

Returns

Array of objects of type [item](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getPartnerID()

Gets the Partner ID of the leadsource parameter from the current session.

Parameters

No parameters to set.

Returns

`partnerid` {String}

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getPasswordHint(customerEmail)

Retrieves password hint for customer with given email address.



Important: Password hints are no longer necessary. The password retrieval email message includes a URL with a password reset code. Therefore, the getPasswordHint() method has been removed as of NetSuite Release 2017.2. As of 2017.2, the method will return an empty string, but generate no errors.

Parameters

- customerEmail [required] {String}

Returns

An empty string

Supported Domains

Checkout

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getPaymentMethod(paymentMethodId, fields)

Gets payment method for given ID.

Parameters

- paymentMethodId [required] {String}
- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned

Returns

Object of type [paymentmethod](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getPaymentMethods(fields)

Gets payment methods accepted by the web store.

Parameters

- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned

Returns

Array of objects of type [paymentmethod](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getRedirectURL()

Gets the URL which is the result of a redirect given the input. Returns null if there is no redirect. Wildcards are also supported (*).

Parameters

- `url` [required] {String}

Returns

String

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getRelatedItems(item)

Gets array of related items for given item.



Note: Complete details are returned for a related item only if the item is:

- available in the store from which the API was called
- available in personalized catalog view for the customer (if Personalized Catalog Views is enabled - for more information, see the help topic [Personalized Catalog Views](#))

Parameters

- `item` [required]{object with values for fields}
 - `internalid` [required]

- itemType [required]
- itemfields [optional] {array of item field names to query}

Note: For better performance limit the number of queried fields by specifying only fields you need in the itemfields array.

Returns

Array of objects of type [item](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getRelatedCorrelatedItems()

Gets a list of related or correlated items, or both, for a given item ID.

Note: Complete details are returned for a related item only if the item is:

- available in the store from which the API was called
- available in personalized catalog view for the customer (if Personalized Catalog Views is enabled - for more information, see the help topic [Personalized Catalog Views](#))

Parameters

- type: [required]{string} Available options are RELATED, CORRELATED, or BOTH.
- fieldsetid: set of fields returned.
- items: list of item IDs. When not specified, items from the cart are returned.
- limit: Maximum count of returned items. When not specified, the default 10 is used. -1 sets no limit.

Important: Use caution when setting the limit to -1 as this can have an impact on performance.

Example

```
1 function service(request, response)
2 {
3   var session = nlapiGetWebContainer().getShoppingSession();
4   items = ['101'];
5   response.writeln("Related Items: " + JSON.stringify(session.getRelatedCorrelatedItem('RELATED', 'relateditems', items, 10)));
6   response.writeln("Correlated Items: " + JSON.stringify(session.getRelatedCorrelatedItems('CORRELATED', 'relateditems',
7   items, 10)));
8   response.writeln("Related & Correlated Items: " + JSON.stringify(session.getRelatedCorrelatedItems('BOTH', 'relateditems',
9   items, 10)));
10 }
```

```
8 | }
```

Returns

Array of correlated or related items.

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getShipToCountries()

Gets countries to which web store can ship.

Parameters

No parameters to set.

Returns

Array of objects with fields:

- name{string}
- code{string}
- isziprequired{boolean}

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [ShoppingSession Methods](#) | **Back to** [Shopping Objects](#)

getShopperCurrency()

Gets currency for current shopping session.

Parameters

No parameters to set.

Returns

Object with values for fields:

- internalid {String}
- name {String}
- symbol {String}
- precision {Number}

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [ShoppingSession Methods](#) | **Back to** [Shopping Objects](#)

getShopperLanguageLocale()

Gets language and locale for current shopping session.

Parameters

No parameters to set.

Returns

localeid {String}

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [ShoppingSession Methods](#) | **Back to** [Shopping Objects](#)

getShopperPreferences()

Gets language, subsidiary, and currency preferences for the current shopping session.

Parameters

No parameters to set.

Returns

Object with values for fields:

- language {String}
- subsidiary {String}
- currency {String}

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getShopperSubsidiary()

Gets subsidiary for current shopping session.

Parameters

No parameters to set.

Returns

subsidiaryid {String}

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getSiteCategoryContents(siteCategory, recursive)

Important: The `getSiteCategoryContents()` API has been removed as of NetSuite Release 2014.2. Use the `getSiteCategoryTree()` method instead. See [getSiteCategoryTree\(siteCategory\)](#).

Gets contents of given site category.

If not provided, `siteCategory` will be defaulted to the root category, and `recursive` will be defaulted to false. When `recursive` is set to false, the API returns the category contents that include subcategories and items; when `recursive` is set to true, the API returns the complete tree of subcategories but not items.



Note: To get the complete category tree from the root, use `getSiteCategoryContents(true)`.

Parameters

- `siteCategory` [optional] {Array} Each object in array has values for field: `internalid` [required]
- `recursive` [optional]

Returns

Array of objects of type [sitecategory](#) or [item](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getSiteCategoryFieldValues(siteCategories)

Gets attribute values of given site categories.

Parameters

- siteCategories [required] {Array} Each object in array has values for field:
 - internalid [required]

Returns

Array of objects of type [sitecategory](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getSiteCategoryTree(siteCategory)

When the internalid attribute is present in the siteCategory object, then the method returns the category sub-tree from this category. When the internalid attribute is not specified, then the whole category tree of the specified site is returned.

When calling this API, you must specify which fields to return. When no fields are specified, then no fields are returned except the internalids of categories. Specify fields using the itemfields attribute on the siteCategory object.

Examples

- The following example returns the entire category tree for the current site. Each category record contains the fields internalid, pagetitle2, and urlcomponent.

```
1 | getSiteCategoryTree({ itemfields : ["pagetitle2","urlcomponent"] });
```

- The following example returns the category sub-tree of category 65. Each category record contains only the internalid field.

```
1 | getSiteCategoryTree({ id : 65 });
```

 **Note:** The examples above return only the category tree, not the category contents.

Parameters

- `siteCategory` [optional] {Array} Each object in array has values for field: `internalid` [required]

Returns

Array of objects of type [sitecategory](#)

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getSiteSettings(fields)

Gets settings of current web store.

Parameters

- `fields` [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned

Returns

Object with fields listed for [sitesettings](#) JSON object.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getSSPApplication()

Gets URL root of SSP application powering the current touchpoint.

Parameters

No parameters to set.

Returns

`urlroot` field value for [sitesettings](#) JSON object.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

getStates(countries)

Gets states and provinces.

Parameters

- countries [optional] Array of strings for country codes

Returns

Returns an array of state/province names and state/province codes in the following format:

```
{ "states": [ { "name": "Alabama", "code": "AL" }, { "name": "Alaska", "code": "AK" }, ... ], "countrycode": "US" }
```

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

isChangePasswordRequest()

Returns true if all page parameters from the current request are valid for the password reset.

Throws the error "ERR_WS_INVALID_LINK" if all page parameters from the current request contain required parameters for password reset but the link has expired or is incomplete.

Parameters

No parameters to set.

Returns

Boolean

Supported Domains

Checkout

Login Required?

no

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

isLoggedIn()

Checks whether the customer has logged in. This method can be used with the `isRecognized()` method. See [Using isRecognized\(\) with isLoggedIn3\(\)](#).



Important: This method is no longer supported. For new Commerce implementations, use the [isLoggedIn3\(\)](#) method. The `isLoggedIn()` method works correctly in the Checkout domain but may not work when used in the Shopping domain even when the user is logged in. If you encounter session issues when using this API, contact support for assistance to migrate to the `isLoggedIn3()` method.

Parameters

No parameters to set.

Returns

Boolean

Supported Domains

Checkout

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

isLoggedIn2()

Checks whether the customer has logged in. This method can be used with the `isRecognized()` method. See [Using isRecognized\(\) with isLoggedIn3\(\)](#).



Important: This method is no longer supported. For new Commerce implementations, use the [isLoggedIn3\(\)](#) method. The `isLoggedIn2()` method works correctly in the Checkout domain but may not work correctly when used with elevated permissions. If you encounter session issues when using this API, contact support for assistance to migrate to the `isLoggedIn3()` method.

Parameters

No parameters to set.

Returns

Boolean

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [ShoppingSession Methods](#) | **Back to** [Shopping Objects](#)

isLoggedIn3()

Checks whether the customer has logged in. This method can be used with the `isRecognized()` method. See [Using isRecognized\(\) with isLoggedIn3\(\)](#).



Important: This method should be used instead of the previous `isLoggedIn()` method and the `isLoggedIn2()` method.

Parameters

No parameters to set.

Returns

Boolean

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [ShoppingSession Methods](#) | **Back to** [Shopping Objects](#)

isRecognized()

Determines whether a user is recognized. This method can be used with the `isLoggedIn2()` and `isLoggedIn3()` methods. However, `isLoggedIn3()` is the preferred method. See [Using isRecognized\(\) with isLoggedIn3\(\)](#).

Parameters

No parameters to set.

Returns

Boolean

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [ShoppingSession Methods](#) | **Back to** [Shopping Objects](#)

logIn(customer)

Logs in a customer.

Parameters

- customer [required]{object with values for fields}
 - email [required]
 - password [required]

Returns

Object with fields:

- customerId {String} - internal id of customer record for logged in customer
- redirecturl {String} - URL to which user is redirected after logging in

Supported Domains

Checkout

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

logout()

Logs out a customer.

 **Note:** If logout() is attempted from a shopping context, it does not succeed and an error is returned.

Parameters

No parameters to set.

Returns

Object with field:

- redirecturl

The field redirecturl has the complete home URL to redirect to after logout. If not using the redirecturl field returned from logout(), the custom redirecturl should have the logoff=T parameter.

Supported Domains

Checkout

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

proceedToCheckout(checkoutSettings)

Used for integration with a third party checkout provider such as PayPal Express. This API can only be called from a secure scheme (https).

Parameters

- checkoutSettings Object with values for fields:
 - type; either "paypal" or "google"
 - continueurl
 - cancelurl

Returns

No value returned.

Supported Domains

Checkout

 **Note:** Must have at least one order.

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

registerCustomer(customer)

Registers the customer and logs them in.

Parameters

- customer [required]{object with values for fields}
 - firstname [required] {String}
 - lastname [required] {String}
 - companyname [optional] {String}
 - email [required] {String}
 - password [required] {String}
 - password2 [required] {String}
 - passwordhint [optional] {String}
 - emailsubscribe [optional] {String}
 - leadsource [optional] {String}

Returns

Object with values for fields:

- `customerid {String}` - internal id of record for registered customer
- `redirecturl {String}` - URL to which user is redirected after registering

Supported Domains

Checkout

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

registerGuest(guest)

Registers a guest.

 **Note:** Throws exception if register guest with password.

Parameters

- `guest [optional]{object with values for fields}`
 - `firstname [optional] {String}`
 - `lastname [required] {String}`

Returns

Object with values for fields:

- `customerid {String}` - internal id of record for registered customer
- `redirecturl {String}` - URL to which user is redirected after registering

Supported Domains

Checkout

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

sendPasswordRetrievalEmail(customeremail)

Sends password retrieval email message to the given email address.

 **Important:** The password reset link generated by the `sendPasswordRetrievalEmail()` includes the customer's email address. Therefore the method has been removed as of NetSuite Release 2017.2. Use the [sendPasswordRetrievalEmail2\(customeremail\)](#) method instead.

Parameters

- `customerEmail [required] {String}`

Returns

No value returned.

Supported Domains

Checkout

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

sendPasswordRetrievalEmail2(customeremail)

Sends password retrieval email message to the given email address.

 **Note:** This method does not include the customer's email address in the reset password link. Use this method instead of the previous method, [sendPasswordRetrievalEmail\(customeremail\)](#).

Parameters

■ customerEmail [required] {String}

Returns

No value returned.

Supported Domains

Checkout

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

setShopperCurrency(currencyid)

Sets currency for current shopping session

 **Note:** Multiple Currencies feature must be enabled, specified currency must be supported in [sitesettings](#), and logged in user must have permission to change currency.

Parameters

■ currencyid [required] {string}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

setShopperLanguageLocale(localeid)

Sets language and locale for current shopping session



Note: Multi-Language feature must be enabled, and specified language must be supported in [sitesettings](#).

Parameters

- localeid [required]{String}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

setShopperPreferences(prefs)

Sets language, subsidiary and currency for current shopping session.

Parameters

- prefs [required]{object with values for fields}
 - language [optional] {String}
 - subsidiary [optional] {String}
 - currency [optional] {String}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

setShopperSubsidiary(subsidiary)

Sets subsidiary for current shopping session (for NetSuite OneWorld users).



Important: The subsidiary set in a session is **not** currently supported out-of-the-box for One World in the SuiteCommerce Advanced SuiteApp. For the SuiteCommerce Advanced SuiteApp, further customization is necessary. Also, the subsidiary for an existing recognized customer can **not** be changed using this API.

Parameters

- subsidiary [required]{String}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

verifyEmailChange(key)

Takes an email change key, verifies it, and then changes the email. If the request can't be processed, an exception is thrown.

Parameters

- key [required]{String} — value of the email change key.

Returns

Boolean

Returns `true` if the key value is valid and the email has been changed. Returns `false` in all other cases (for example, if the key is invalid or expired).

Supported Domains

Checkout

Login Required?

Yes

[Back to ShoppingSession Methods](#) | [Back to Shopping Objects](#)

Using isRecognized() with isLoggedIn3()

There are four possible scenarios for using the `isLoggedIn3()` method with the `isRecognized()` method to correctly determine if a user is logged in:

1. New (anonymous) user: `isRecognized() == false` and `isLoggedIn3() == false`
2. Registered and logged-in user: `isRecognized() == true` and `isLoggedIn3() == true`
3. User clicked the logout link: `isRecognized() == false` and `isLoggedIn3() == false`
4. User logged in, then closed the browser and later returned to your Commerce store: `isRecognized() == true` and `isLoggedIn3() == false`

Note: The fourth scenario is the only case where the `isRecognized()` status differs from the `isLoggedIn3()` status. In this case, the user did not click on the logout link and is therefore still recognized but is not logged in.

Customer Methods

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

A customer object holds data for the logged in customer. This object is different from a NetSuite customer record, because it holds information for the given shopping session only.

The following Customer methods are available:

- `addAddress(address)`
- `addCreditCard(creditcard)`
- `emailCustomer(subject, body)`
- `getAddress(addressid, fields)`
- `getAddressBook(fields)`
- `getCampaignSubscriptions(fields)`
- `getCampaignSubscriptions(subscriptionId,fields)`
- `getCreditCard(creditcardid, fields)`
- `getCreditCards(fields)`
- `getCustomerSegments()`
- `getCustomFields()`
- `getCustomFieldValues()`
- `getFieldValues(fields)`
- `isGuest()`
- `removeAddress(addressid)`
- `removeCreditCard(creditcardid)`
- `setLoginCredentials(customer)`
- `updateAddress(address)`
- `updateCampaignSubscriptions(subscriptions)`

- [updateCreditCard\(creditcard\)](#)
- [updateProfile\(customer\)](#)

addAddress(address)

Adds an address for current customer.

Parameters

- address [required] {Object with values for fields}
 - addressee [required]
 - addr1 [required]
 - addr2 [optional]
 - addr3 [optional]
 - city [required]
 - state [required]
 - country [required]
 - zip [required]
 - phone [optional]
 - isresidential [optional]
 - defaultshipping [optional]
 - defaultbilling [optional]

Returns

String - key of added address.

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

addCreditCard(creditcard)

Adds a credit card for current customer.

 **Note:** This method is not compatible with SuiteCommerce 2019.1 if the Payments Instruments feature is enabled. SuiteScript methods should be used instead.

Parameters

- creditcard [required] {Object with values for fields}
 - ccnumber [required]

- ccname [required]
- authcode [optional]
- customercode [optional]
- paymentmethod [required] (numerical ID for credit card type in NetSuite)
- expmonth [required]
- expyear [required]
- validfrommon [required for UK]
- validfromyear [required for UK]
- debtcardissueno [required for UK]
- ccdefault [optional]

Returns

String - key of added credit card.

Supported Domains

Checkout

Login Required?

Yes

Back to [Customer Methods](#) | **Back to** [Shopping Objects](#)

emailCustomer(subject, body)

Sends email to current customer with given subject and body.

Parameters

- subject [required] {String}
- body [required] {String}

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Back to [Customer Methods](#) | **Back to** [Shopping Objects](#)

getAddress(addressid, fields)

Gets address with given ID for current customer.

Parameters

- addressid [required] {string}
- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [address](#).

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

getAddressBook(fields)

Gets all addresses for current customer

Parameters

- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Array of objects of type [address](#).

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

getCampaignSubscriptions(fields)

Gets subscription information for the current customer

Parameters

- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Array of objects of type [subscriptions](#).

Supported Domains

Checkout, Shopping

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)**getCampaignSubscriptions(subscriptionId,fields)**

Gets data for a single campaign subscription with passed id.

Parameters

- subscriptionId [required] {String}
- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned

Returns

Array of objects of type [subscriptions](#)

Supported Domains

Checkout, Shopping

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)**getCreditCard(creditcardid, fields)**

Gets credit card with given ID for current customer.



Note: This method is not compatible with SuiteCommerce 2019.1 if the Payments Instruments feature is enabled. SuiteScript methods should be used instead.

Parameters

- creditcard [required] {Object with values for fields}
 - internalid [required]
 - ccnumber [required]
 - ccname [required]
 - authcode [optional]
 - customercode [optional]
 - paymentmethod [required]
 - expmonth [required]

- expyear [required]
- validfrommon [required for UK]
- validfromyear [required for UK]
- debtcardissueno [required for UK]
- ccdefault [optional]

Returns

Object of type [creditcard](#).

Supported Domains

Checkout

Login Required?

Yes

Back to [Customer Methods](#) | **Back to** [Shopping Objects](#)

getCreditCards(fields)

Gets all credit cards for current customer.



Note: This method is not compatible with SuiteCommerce 2019.1 if the Payments Instruments feature is enabled. SuiteScript methods should be used instead.

Parameters

- customer [required] {Object with values for fields}
 - internalid [required]
 - email [optional]
 - emailsubscribe [optional]
 - firstname [optional]
 - lastname [optional]
 - middlename [optional]
 - phoneinfo [optional]
 - customfields [optional] {Object with {customfield id : customfield value} }

Returns

Array of objects of type [creditcard](#).

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

getCustomerSegments()

Gets all customer segments to which the current customer belongs.

Parameters

No parameters to set.

Returns

Array of objects with the field:

- id {number}

Supported Domains

Shopping, Checkout

Login Required?

No

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

getCustomFields()

Gets custom fields on customer record for current customer.

Parameters

No parameters to set.

Returns

Object with custom field names.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

getCustomFieldValues()

Gets custom field values on customer record for current customer.

Parameters

No parameters to set.

Returns

Object with custom field values. Multiple field values are separated by the Unicode character 'ENQUIRY' ("        ").

Supported Domains

Checkout, Shopping

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

getFieldValues(fields)

Gets standard field values on customer record for current customer.

Parameters

- **fields** [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [customer](#).

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

isGuest()

Checks whether the shopper is logged in as a guest. Returns False when the shopper is not logged in or is logged in as a customer.

Parameters

No parameters to set.

Returns

Boolean

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

removeAddress(addressid)

Removes address with given ID from current customer's list of addresses.

Parameters

- addressid [required] {string}

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

removeCreditCard(creditcardid)

Removes given credit card from current customer.

 **Note:** This method is not compatible with SuiteCommerce 2019.1 if the Payments Instruments feature is enabled. SuiteScript methods should be used instead.

Parameters

- creditcardid [required] {string}

 **Note:** Websites on SuiteCommerce versions earlier than 2019.1 must have a patch applied to be compatible with the Payments Instruments feature. See [link to patch documentation](#). If the correct patch has been applied, the Payment Instruments feature is enabled, and the credit card has been tokenized, the creditcardid parameter should contain the numerical ID of the payment token.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

setLoginCredentials(customer)

Sets login credentials for current guest customer.

Parameters

- customer [required] {Object with values for fields}
 - internalid [required]
 - email [required]
 - password [required]
 - customfields [Optional] {Object with {customfield id : customfield value} }

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

updateAddress(address)

Updates a specified address.

Parameters

- address [required] {Object with values for fields}
 - internalid [required]
 - addressee [required]
 - addr1 [required]
 - addr2 [optional]
 - addr3 [optional]
 - city [required]
 - state [required]
 - country [required]
 - zip [required]
 - phone [optional]
 - isresidential [optional]
 - defaultshipping [optional]
 - defaultbilling [optional]

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

updateCampaignSubscriptions(subscriptions)

Updates a specified campaign subscription.

Parameters

- subscriptions [required] {Object with values for fields}
 - internalid [required]
 - subscribed [required]

Customer can only opt-in or opt-out of an email campaign category by setting subscribed to 'T' or 'F'.

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

updateCreditCard(creditcard)

Updates given credit card for current customer.



Note: This method is not compatible with SuiteCommerce 2019.1 if the Payments Instruments feature is enabled.

Parameters

- creditcard [required] {Object with values for fields}
 - internalid [required]
 - ccnumber [required]

- ccname [required]
- authcode [optional]
- customercode [optional]
- paymentmethod [required]
- expmonth [required]
- expyear [required]
- validfrommon [required for UK]
- validfromyear [required for UK]
- debtcardissueno [required for UK]
- ccdefault [optional]



Note: If given ccnumber does not match an existing record, internalid is ignored and a new credit card record is added for customer.

Returns

String - key of added or updated credit card.

Supported Domains

Checkout

Login Required?

Yes

[Back to Customer Methods](#) | [Back to Shopping Objects](#)

updateProfile(customer)

Updates profile of current customer.

Parameters

- customer [required] {Object with values for fields}
 - internalid [required]
 - email [optional]
 - emailsubscribe [optional]
 - firstname [optional]
 - lastname [optional]
 - middlename [optional]
 - phoneinfo [optional]
 - customfields [optional]{Object with {customfield id : customfield value} }
 - cookieoptions [optional]{Object with {cookie approval type : boolean value} }

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Order Methods

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

An order object holds shopping cart data and methods for placing the order. This object is different from a NetSuite order record, because it holds information for the given shopping session only.

The following Order methods are available:

- `addFreeGiftItem(item)`
- `addItem(item)`
- `addItems(items)`
- `applyGiftCertificate(giftcertificate)`
- `applyPromotionCode(promocode)`
- `estimateShippingCost(address)`
- `setItemExcludedFromShipping(cartItemId,excluded)`
- `getAppliedGiftCertificate(giftcertificate, fields)`
- `getAppliedGiftCertificates(fields)`
- `getAppliedPromotionCode(promocode, fields)`
- `getAppliedPromotionCodes(fields)`
- `getAvailableShippingMethod(shipmethodid,fields)`
- `getAvailableShippingMethods(orderLineIds, shipAddressId)`
- `getAvailableShippingMethods(fields)`
- `getBillingAddress()`
- `getCustomFields()`
- `getCustomFieldValues()`
- `getDeviceFingerPrintingHtml()`
- `getEligibleFreeGiftItems()`
- `getFieldValues(fields)`
- `getItem(orderitemid, fields)`
- `getItemOption(orderitemid,fields)`
- `getItems(bConsolidated), getItems(fields, bConsolidated)`
- `getItems(fields)`
- `getItemShippingFieldValues(orderitemid)`
- `getOrderSummary(fields)`
- `getPayment(fields)`

- `getPurchaseNumber()`
- `getShippingAddress()`
- `getShippingMethod(fields)`
- `getSummaryTaxTotals()`
- `getTaxDetails(), getTaxDetails(fields)`
- `mergeItems(orderLineIds)`
- `recalculateTaxes()`
- `rejectFreeGiftItem(item)`
- `removeAllItems()`
- `removeBillingAddress()`
- `removeItem(orderitemid)`
- `removePayment()`
- `removePurchaseNumber()`
- `removePromotionCode(promocode)`
- `removeAllPromotionCodes()`
- `removeShippingAddress()`
- `removeShippingMethod()`
- `setBillingAddress(addressid)`
- `setCardHolderAuthentication(cardHolderAuthentication)`
- `setCustomFieldValues(customfields)`
- `setEnableItemLineShipping()`
- `setItemOptions(itemlineid,optionMap)`
- `setItemShippingAddress(orderItemId, shipAddressId)`
- `setItemShippingAddress(orderItemIds, shipAddressId)`
- `setItemShippingMethod(orderItemId, shipMethodId)`
- `setItemShippingMethod(orderItemIds, shipMethodId)`
- `setPayment(payment)`
- `setPurchaseNumber(purchaseNumber)`
- `setShippingAddress(addressid)`
- `setShippingMethod(shipmethod)`
- `setTermsAndConditions(readandagree)`
- `splititem(orderLine)`
- `submit(ordersettings)`
- `updateItemFulfillmentPreferences(item)`
- `updateItemsFulfillmentPreferences(items)`
- `updateItemQuantity(item)`
- `updateItemQuantities(items)`

addFreeGiftItem(item)

Adds the eligible item to the cart and marks it as a free item.

Parameters

- `item` [required] {Object with values for fields}
 - `item_id` [required] {Number}
 - `promotion_id` [required] {Number}
 - `quantity` [required] {Number or String}
 - `options` [required for item with item options]

Note: *quantity* can be a negative number.

Returns

String - the `orderid` of added item.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

`addItem(item)`

Adds an item to order.

 **Note:** If your selected Out Of Stock Behavior option is **Disallow back orders but display out-of-stock message** or **Remove out-of-stock items from store**, you might still be able to add an out of stock item to your cart using the `addItem(item)` order method. However, when you try to place the order with such out of stock item in your shopping cart, an error message is displayed and you will not be able to place the order successfully.

Parameters

- `item` [required] {Object with values for fields}
 - `internalid` [required]
 - `quantity` [required] {Number or String}
 - `options` [required for item with item options]
 - `fulfillmentPreferences` [optional] {fulfillmentPreferences object}

Note: *quantity* can be a negative number.

Returns

String - the `orderid` of added item.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

addItem(items)

Adds multiple items to cart/order within a single request.



Note: If your selected Out Of Stock Behavior option is **Disallow back orders but display out-of-stock message** or **Remove out-of-stock items from store**, you might still be able to add out of stock items to your cart using the addItem(items) order method. However, when you try to place the order with such out of stock items in your shopping cart, an error message is displayed and you will not be able to place the order successfully.

Parameters

- items [required]{Array of objects with values for fields}
 - internalid [required]
 - quantity [required] {Number or String}
 - options [required for item with item options]
 - fulfillmentPreferences [optional] {fulfillmentPreferences object}

Returns

String - the orderitemid of added items.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

applyGiftCertificate(giftcertificate)

Applies given gift certificate to the order.

Parameters

- giftcertificate [required] {String}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

applyPromotionCode(promocode)

Applies given promotion code to the order.

Parameters

- promocode [required] {String}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

estimateShippingCost(address)

Gets estimated shipping cost for given address.



Note: Estimated shipping cost depends on order's shipping method, enabling of shipping estimator on site administration page, and whether order requires shipping.

Parameters

- address [optional] {Object with fields}
 - zip [required]
 - country [required]
 - state [optional]

Returns

Number

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setItemExcludedFromShipping(cartItemId,excluded)

Identifies when an item on an order is excluded from shipping.

Parameters

- cartItemId [required] {string}
- excluded [required] {boolean}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getAppliedGiftCertificate(giftcertificate, fields)

Gets specified gift certificate.

Parameters

- giftCertificate [required] Certificate(giftcertificate, fields)
- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [giftcertificate](#).

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getAppliedGiftCertificates(fields)

Gets all applied gift certificates.

Parameters

- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Array of objects of type [giftcertificate](#)

Supported Domains

Checkout

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getAppliedPromotionCode(promocode, fields)

Gets specified promotion code.

Parameters

- `promoCode` [required] {String}
- `fields` [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [promocode](#)

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getAppliedPromotionCodes(fields)

Gets all applied promotion codes.

Parameters

- `fields` [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Array of objects of type [promocode](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getAvailableShippingMethods(orderLineIds, shipAddressId)

Returns available shipping methods and addresses for the given item in the order.

 **Note:** The `setEnabledItemLineShipping()` flag must be set before using this method. See [setEnabledItemLineShipping\(\)](#).

Parameters

- `orderLineIds` [required] Array of `orderitemids` to merge
- `shipAddressId` [required] String

Returns

Array of objects of type [shipmethod](#).

Supported Domains

Checkout

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getAvailableShippingMethod(shipmethodid,fields)

Gets specified shipping method.

Parameters

- `shipmethodid` [required] {String}
- `fields` [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [shipmethod](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getAvailableShippingMethods(fields)

Gets all available shipping methods.

Parameters

- **fields** [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Array of objects of type [shipmethod](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getBillingAddress()

Gets billing address for order.

Parameters

No parameters to set.

Returns

Object of type [address](#).

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getCustomFields()

Gets custom fields on order record.

Parameters

No parameters to set.

Returns

Object with custom field names.

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getCustomFieldValues()

Gets custom field values for current order.

Parameters

No parameters to set.

Returns

Object with custom field values.

Supported Domains

Checkout, Shopping

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getDeviceFingerprintingHtml()

Sets device finger printing for Cybersource for the device a shopper is logged in from when accessing a web store. This can be used with the Cybersource payment gateway for added site security. The API call must be made in the last step of checkout. For more details on device fingerprinting in Commerce web stores, see the help topic [Device Fingerprinting](#).

Parameters

No parameters to set.

Returns

Returns an HTML string that needs to be rendered on the last page of a checkout that includes the Submit Order button.

Supported Domains

Checkout

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getEligibleFreeGiftItems()

Gets a list of all Free Item promotions that are currently applied. It also gets the Item IDs of all the free items.

Parameters

No parameters to set.

Returns

Array of objects with the following fields:

- promotion_id
- eligible_items
- eligible_quantity
- rejected_quantity
- added_quantity

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getFieldValues(fields)

Gets standard field values for the current order.

Parameters

- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [order](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getItem(orderitemid, fields)

Gets specified order item.

Parameters

- orderitemid [required] {String}
- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [orderitem](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getItemOption(orderitemid,fields)

Gets item option for specified order item.

Parameters

- orderitemid [required] {String}
- fields [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

The options field from the object of type [orderitem](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getItems(bConsolidated), getItems(fields, bConsolidated)

Used to configure multiple shipping routes on web orders. Depending on the value for bConsolidated, order items are returned in either a split view or in a consolidated view.

For example, displaying an unconsolidated view of all the items in the cart allows shoppers to select the shipping address for each item. You can then use the consolidated view to display items to be shipped, grouped by shipping address.

 **Note:** The `setEnableItemLineShipping()` flag must be set before using this method. See [setEnableItemLineShipping\(\)](#).

Parameters

- `bConsolidated` Boolean
- `fields` Array of field names to be included in returned JSON object; if omitted, all supported fields are returned

Returns

Object of type [orderitem](#).

Supported Domains

Checkout

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getItems(fields)

Gets order items.

Parameters

- `fields` [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Array of objects of type [orderitem](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getItemShippingFieldValues(orderitemid)

Gets shipping field values for specified order item. This API is only available when the Multiple Shipping Routes feature is enabled.

Parameters

- `orderid` [required] {String}

Returns

Object with fields:

- `shipaddress`
- `shipmethod`

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getOrderSummary(fields)

Gets the order summary.



Important: For the most optimized results, use the `itemcount` argument with this method. When this method is called with no arguments, all information about the order is returned which results in poor performance.

Example

```
1 | var itemcount = nlapiGetWebContainer().getShoppingSession().getOrder().getOrderSummary(['itemcount']);
```

Parameters

- `fields` [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned

Returns

Object of type [ordersummary](#).

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

getPayment(fields)

Gets payment details for order.

Parameters

- **fields** [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [payment](#).

Supported Domains

Checkout

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getPurchaseNumber()

Gets the Purchase order number associated with the order.

Parameters

No parameters to set.

Returns

String

Supported Domains

Checkout

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getShippingAddress()

Gets shipping address for order.

Parameters

No parameters to set.

Returns

Object of type [address](#)

Supported Domains

Checkout

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getShippingMethod(fields)

Gets shipping method for order.

Parameters

- `fields` [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned}

Returns

Object of type [shipmethod](#).

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getSummaryTaxTotals()

Gets details of the tax amount for each type of tax applicable for a sales order. This method works only when the SuiteTax feature is enabled. For information about SuiteTax, see the help topics [SuiteTax](#), [Working with Taxes using Commerce API](#) and [SuiteCommerce SuiteTax Support](#).

Parameters

No parameters to set.

Returns

- If SuiteTax is enabled, returns object of type [summarytax](#).
- If SuiteTax is not enabled, returns error message.

Supported Domains

Checkout, Shopping

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

getTaxDetails(), getTaxDetails(fields)

Gets details of tax applicable for each item in the sales order. Use these methods only if the SuiteTax feature is enabled. For information about SuiteTax, see the help topics [SuiteTax](#), [Working with Taxes using Commerce API](#) and [SuiteCommerce SuiteTax Support](#).

Parameters

- **fields** [optional] Array of field names to be included in returned JSON object; if omitted, all supported fields are returned

Returns

- If SuiteTax is enabled, returns object of type [taxdetails](#).
If handling is set up as separate from shipping, returns separate taxdetails objects for shipping and handling. If multiple tax types are applicable to the order, returns one taxdetails object per tax type per item.
- If SuiteTax is not enabled, returns error message.

Supported Domains

Checkout, Shopping

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

mergeItems(orderLineIds)

Merges the given line items into a single order line item. The given order lines can only differ in quantity and shipping address and method.

 **Note:** The `setEnabledItemLineShipping()` flag must be set before using this method. See [setEnabledItemLineShipping\(\)](#).

Parameters

- **orderLineIds**[required]: Array of `orderitemids` to merge

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Sample Code

```
1 | nlapiGetWebContainer().getShoppingSession().getOrder().mergeItems(orderLineIds);
```

[Back to Order Methods](#) | [Back to Shopping Objects](#)

recalculateTaxes()

Triggers tax calculation for a sales order. Use this method only if the SuiteTax feature is enabled.

You must call this method to calculate taxes for a sales order every time you need updated tax details. Taxes are not calculated or recalculated unless this method is called.

SuiteTax uses external tax engines to calculate taxes and this can have an impact on performance. You should limit the use of this method to situations where tax recalculation is necessary. For information about SuiteTax, see the help topics [SuiteTax](#), [Working with Taxes using Commerce API](#) and [SuiteCommerce SuiteTax Support](#).

Parameters

No parameters to set.

Returns

- If calculation is successful, does not return a value.
- If calculation fails, returns error message.
- If SuiteTax is not enabled, returns error message.

Supported Domains

Checkout, Shopping

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

rejectFreeGiftItem(item)

Allows shoppers to reject the free item provided the item is not already in the cart.

Parameters

- `item` [required] {Object with values for fields}
 - `promotion_id` [required] {Number}
 - `quantity` [required] {Number or String}

Note: *quantity* can be a negative number.

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

removeAllGiftCertificates()

Removes all gift certificates from an order.

Parameters

No parameters to set.

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

Sample Code

```
1 | nLapiGetWebContainer().getShoppingSession().getOrder().removeAllGiftCertificates()
```

[Back to Order Methods](#) | [Back to Shopping Objects](#)

removeAllItems()

Removes all items from order.

Parameters

No parameters to set.

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

removeBillingAddress()

Removes the current billing address from the order.

Parameters

No parameters to set.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

removeItem(orderitemid)

Removes specified item from order.

Parameters

- orderitemid [required] {String}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

removeItemOptions(itemlineid)

Removes specified item options from a row in the cart.

Parameters

- itemlineid [required]

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

removePayment()

Removes the current payment method for the order.

Parameters

No parameters to set.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

removePurchaseNumber()

Removes the associated Purchase order number from order.

Parameters

No parameters to set.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

removePromotionCode(promocode)

Removes specified promotion code from order.

Parameters

- `promocode` [required] {String}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

removeAllPromotionCodes()

Removes all promotion codes from the order.

Parameters

No parameters to set.

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

removeShippingAddress()

Removes the current shipping address from the order.

Parameters

No parameters to set.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

removeShippingMethod()

Removes the current shipping method for the order.

Parameters

No parameters to set.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

setBillingAddress(addressid)

Sets the billing address for the order.

Parameters

■ addressid [required] {String}

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

setCardHolderAuthentication(cardHolderAuthentication)

Sets the credit card holder authentication for the order.

Parameters

- cardHolderAuthentication[required] {String}

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setCustomFieldValues(customfields)

Sets custom field values for order.

Parameters

- customfields [required] {Object with name/value pair for each field}



Important: Custom fields of the type **Document** are NOT supported for non-employee external centers.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setEnabledItemLineShipping()

Toggles the item line shipping flag for the current order. This flag **must** be set prior to using other Multiple Ship To APIs.



Important: The Multiple Shipping Routes feature is required.

An exception is thrown when:

- The Multiple Shipping Routes feature is not enabled
- The site is not using the Standard Sales Order Form for scripting

Parameters

- enable [required]: Boolean

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Sample Code

```
1 | nlapiGetWebContainer().getShoppingSession().getOrder().setEnabledItemLineShipping(true);
```

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setItemOptions(itemlineid,optionMap)

Updates the item options for a particular cart row.

Parameters

- itemlineid [required]
- optionMap [required] {Array of Item Option ID value pairs}

Note: itemlineid is a combination of item id and set. This is included because of the possibility for multiple quantities of the same item but with different item options.

Returns

itemLineId of the new line with the new item options.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setItemShippingAddress(orderItemId, shipAddressId)

Sets the shipping address for the given order line item.

Note: The `setEnabledItemLineShipping()` flag must be set before using this method. See [setEnabledItemLineShipping\(\)](#).

Parameters

- `orderIdId[required]`: String
- `shipAddressId[required]`: String

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Sample Code

```
1 | nlapiGetWebContainer().getShoppingSession().getOrder().setItemShippingAddress(orderItemId, shipAddressId);
```

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setItemShippingAddress(orderItemIds, shipAddressId)

Sets the shipping address for an array of given order line items.

Note: The `setEnabledItemLineShipping()` flag must be set before using this method. See [setEnabledItemLineShipping\(\)](#).

Parameters

- `orderItemIds[required]`: Array of `orderIdId` fields.
- `shipAddressId[required]`: String

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setItemShippingMethod(orderItemId, shipMethodId)

Sets the shipping method for the given order line item.

Note: The `setEnabledItemLineShipping()` flag must be set before using this method. See [setEnabledItemLineShipping\(\)](#).

Parameters

- `orderItemId[required]`: String.
- `shipMethodId[required]`: String

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Sample Code

```
1 | nLapiGetWebContainer().getShoppingSession().getOrder().setItemShippingMethod(orderItemId, shipMethodId);
```

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setItemShippingMethod(orderItemIds, shipMethodId)

Sets the shipping method for an array of given order line items.

Note: The `setEnabledItemLineShipping()` flag must be set before using this method. See [setEnabledItemLineShipping\(\)](#).

Parameters

- `orderItemIds[required]`: Array of `orderItemId` fields.
- `shipMethodId[required]`: String

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setPayment(payment)

Sets payment for order; for credit card, creates a new record if internalid not provided.

Parameters

- payment [required] {Object with values for fields}
 - internalid [optional]
 - *ccname [optional]
 - *ccnumber [optional]
 - *expmonth [optional]
 - *expyear [optional]
 - paymentmethod [required] {If Payment Instruments feature is enabled, this contains the numerical ID of payment card token}
 - paymentterms[optional]
- creditcard [optional] (this is an object within the payment object.)
- ach [optional] (this is an object within the payment object.)



Note: * Fields are required only for credit card payment methods.

For information about creating non-credit card payment methods, see the help topics [Alternative Payment Methods](#) and [ACH Payments](#).

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setPurchaseNumber(purchaseNumber)

Sets the purchase number for the order.

Parameters

- purchaseNumber [required] {String}

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setShippingAddress(addressid)

Sets the shipping address for the order.

Parameters

- addressid [required] {String}

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setShippingMethod(shipmethod)

Sets shipping method for order.

Parameters

- shipmethod [required] {Object with values for fields}
 - shipmethod [required]
 - shipcarrier [required]



Note: Valid values that can be set for shipcarrier are ups or nups.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

setTermsAndConditions(readandagree)

If required by site, sets whether user has read and agreed to terms and conditions.

Parameters

- readandagree [required] {Boolean}, or {String}, where value must be "T" or "F"

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

splititem(orderLine)

Splits the given line item into multiple order line items. You can then set a shipping address for each order line.



Note: The `setEnabledItemLineShipping()` flag must be set before using this method. See [setEnabledItemLineShipping\(\)](#).

Parameters

- orderLine [required] {Object with values for fields}
 - orderitemid [required]
 - quantities [optional] An array of quantities to split into. If not provided, all of the line items from the split are given a quantity of 1.

Returns

No value returned.

Supported Domains

Checkout

Login Required?

Yes

Sample Code

```
1 | nLapiGetWebContainer().getShoppingSession().getOrder().splitItem(orderitem);
```

[Back to Order Methods](#) | [Back to Shopping Objects](#)

submit(ordersettings)

Places the shopping order.

Parameters

- [checkoutsettings](#)

Returns

Object with fields:

- status
- internalid
- confirmationnumber

Supported Domains

Checkout

Login Required?

Yes

[Back to Order Methods](#) | [Back to Shopping Objects](#)

updateItemFulfillmentPreferences(item)

Updates specified item's fulfillment choices for the order.

Parameters

- item [required] {Object with values for fields}
 - orderitemid [required]
 - fulfillmentPreferences [required] {fulfillmentPreferences object}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

[Back to Order Methods](#) | [Back to Shopping Objects](#)

updateItemsFulfillmentPreferences(items)

Updates specified items' fulfillment choices for the order.

Parameters

- `items` [required] {Array of objects with values for fields}
 - `orderid` [required]
 - `fulfillmentPreferences` [required] {fulfillmentPreferences object}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

updateItemQuantity(item)

Updates specified item's quantity for order.

Parameters

- `item` [required]{Object with values for fields}
 - `orderid` [required]
 - `quantity` [required] {Number or String}

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

Back to [Order Methods](#) | **Back to** [Shopping Objects](#)

updateItemQuantities(items)

Updates specified items' quantities for order.

Parameters

- `items` [required]{Array of objects with values for fields}
 - `orderid` [required]

- quantity [optional] {Number or String} Defaults to 1 if omitted.

Returns

No value returned.

Supported Domains

Checkout, Shopping

Login Required?

No

Understanding Multiple Ship To

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

APIs for Multiple Ship To provide the ability to code a checkout experience that allows shoppers to ship items in their cart to multiple shipping addresses.

Before working with the Multiple Ship To APIs, ensure the **Multiple Shipping Routes** feature is enabled in your account. For detailed information about the Multiple Shipping Routes feature, see the help topic [Multiple Shipping Routes](#).

The set of APIs related to Multiple Ship To include:

- [updateItemFulfillmentPreferences\(item\)](#)
- [updateItemsFulfillmentPreferences\(items\)](#)
- [setEnableItemLineShipping\(\)](#)
- [splititem\(orderLine\)](#)
- [mergeItems\(orderLineIds\)](#)
- [setItemShippingAddress\(orderItemId, shipAddressId\)](#)
- [setItemShippingAddress\(orderItemIds, shipAddressId\)](#)
- [setItemShippingMethod\(orderItemId, shipMethodId\)](#)
- [setItemShippingMethod\(orderItemIds, shipMethodId\)](#)
- [getItems\(bConsolidated\), getItems\(fields, bConsolidated\)](#)
- [getAvailableShippingMethods\(orderLineIds, shipAddressId\)](#)
- [addItem\(item\)](#)
- [addItem\(items\)](#)

Known Limitations

- **Promotions:** Multiple Ship To does not work with promotions with shipping impact. The promo code is automatically removed from the order.
- **One shipping method per shipping address:** Multiple Ship To is limited to defining one shipping method per shipping address. Shoppers can not create multiple shipments to the same address. For example, a shopper can not have one set of items shipping to their Home address using a Standard shipping method and another set of items shipping to their Home address using a One Day shipping method.

- **Real time rates and performance:** There is a known performance impact on the Proceed to Checkout flow when a ship method with real time rates is used. This can be exacerbated for orders with Multiple Ship To. Ensure that you do perform performance testing before going live when using real time rates.
- **Login Required:** To use the above mentioned Order methods for Multiple Ship To, you must be logged in.

Working with Taxes using Commerce API

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

There are two tax calculation features available in NetSuite — SuiteTax and Legacy Taxes. SuiteTax uses internal or external tax engines to calculate taxes for transactions in NetSuite. At present, SuiteTax is available only for Commerce web stores that cater to the US and Canada. See the help topics [SuiteTax](#) and [SuiteCommerce SuiteTax Support](#) for more information about SuiteTax.

Important: SuiteTax is not supported for tax calculation on SiteBuilder websites.

SuiteTax

SuiteTax calculates the total taxes applicable for each item in a sales order. The following methods retrieve SuiteTax tax calculations for a web store order:

- `recalculateTaxes()` - triggers tax calculation for a sales order. Taxes are not calculated or recalculated unless this method is called.
- `getTaxDetails()`, `getTaxDetails(fields)` - retrieves tax details for each line item in the sales order, including items, shipping, and handling (if set up as separate from shipping). If multiple tax types apply to a sales order, this method retrieves a `taxdetails` object for each tax type for each line item.
 - The `taxdetailsreference` field for an item in the `orderitem` object matches the `taxdetailsreference` field in the related `taxdetails` objects returned by `getTaxDetails()`. If `getTaxDetails()` does not return any `taxdetails` objects with a matching `taxdetailsreference` field, it means no taxes are applicable for that item.
 - The `taxamount` field contains the total of the taxes applicable for the item. If multiple taxes apply to a line item, this is the total amount for the tax type in the `taxtype` field.
- `getOrderSummary(fields)` - retrieves the fields:
 - `taxonshipping` - shipping tax amount.
 - `taxtotal` - total taxes for the whole order.
- `getSummaryTaxTotals()` - retrieves the tax amount per tax type for the whole sales order. If multiple tax types apply to a sales order, this method returns multiple `summarytax` objects.

Legacy Taxes

Note: The field used for legacy taxes must not be used if SuiteTax is enabled.

In the United States, per item taxes are not applied to an order. Instead tax is applied to the entire order. The following tax fields are present in the `orderitem` object and are only used when countries require per line tax:

- `taxtype1`

- `taxrate1`
- `taxtype2`
- `taxrate2`

Since every line can have a different tax type or tax rate you should define the taxes per order line. When a line is not taxed, omit those fields.

Note: You should also check the value of `pricesincludevat` in the `sitesettings` object to verify whether taxes are included in the price. These fields are only required when tax is not included in the price.

PageGenerator Methods

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Methods on the `pageGenerator` object are used to generate a web store page.

The following `PageGenerator` methods are available:

- `addBreadCrumb(label, url)`
- `addHeadHtml(html)`
- `addStylesheetHtml(html)`
- `addTab(id, label, url)`
- `addToPageInitScript(scriptLines)`
- `removeBreadCrumbs()`
- `removeTab(id)`
- `removeTabs()`
- `setDocTypeHtml(html)`
- `setMetaTagHtml(html)`
- `setPageTitle(title)`
- `setSelectedTab(id)`
- `showTabNavigation(show)`

addBreadCrumb(label, url)

Adds a bread crumb to the end of the existing bread crumb trail.

Parameters

- `label` [required] {string} - text displayed for the bread crumb
- `url` [required] {string} - URL to which the bread crumb links

Back to [PageGenerator Methods](#) | **Back to** [Shopping Objects](#)

addHeadHtml(html)

Adds the specified HTML to the top of the page's `<head>` section, before the stylesheet HTML.

Parameters

- `html` [required] {string} - HTML content

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

addStylesheetHtml(html)

Adds the specified HTML to the top of the page's <head> section, after the stylesheet HTML

Parameters

- `html` [required] {string} - HTML content

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

addTab(id, label, url)

Adds a tab to the right of current tabs on the page.

Parameters

- `id` [required] {string} - unique string id for the tab
- `label` [required] {string} - label displayed within the tab
- `url` [required] {string} - URL to which the tab links

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

addToPageInitScript(scriptLines)

Adds lines of JavaScript to the page's initialization script.

Parameters

- `scriptLines` [required] {string}

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

removeBreadCrumbs()

Removes all bread crumbs from the page.

Parameters

No parameters to set.

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

removeTab(id)

Removes a specified tab from the page.

Parameters

- `id` [required] {string} - unique string id for the tab

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

removeTabs()

Removes all tabs from the page.

Parameters

No parameters to set.

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

setDocTypeHtml(html)

Establishes HTML for the page <doctype> tag.

Parameters

- `html` [required] {string} - HTML content

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

setMetaTagHtml(html)

Establishes HTML for the page's <meta> tags.

Parameters

- `html` [required] {string} - HTML content

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

setPageTitle(title)

Establishes a title for the page.

Parameters

- `title` [required] {string} - title text

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

setSelectedTab(id)

Causes the specified tab to be displayed as the currently active tab.

Parameters

- `id` [required] {string} - unique string id for the tab

[Back to PageGenerator Methods](#) | [Back to Shopping Objects](#)

showTabNavigation(show)

Indicates whether tab navigation should be shown or hidden.

Parameters

- `show [required] {boolean}` - true to display navigation, false to hide it

StandardTagLibrary Methods

① Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Methods on a `standardTagLibrary` object implement NetSuite web store tags.

Each of the following methods returns the same result as the corresponding NetSuite page tag. All results are strings.

- `getCartLinkHtml()`
- `getCartUrl()`
- `getCheckoutLinkHtml()`
- `getCheckoutUrl()`
- `getCompanyId()`
- `getCrumbTrail()`
- `getCurrencySelectHtml()`
- `getCustomerCenterLinkHtml()`
- `getCustomerCenterUrl()`
- `getGlobalSearchFormHtml()`
- `getGlobalSearchHtml()`
- `getJSessionId()`
- `getLanguageSelectHtml()`
- `getLoginLinkHtml()`
- `getLoginUrl()`
- `getLogoutLinkHtml()`
- `getLogoutUrl()`
- `getPageFullHead()`
- `getPageHead()`
- `getPageLinks()`
- `getPageTabs()`
- `getPageTop()`
- `getRegionSelectHtml()`
- `getRegistrationLinkHtml()`
- `getRegistrationUrl()`
- `getReloginLinkHtml()`
- `getReloginUrl()`
- `getSideBarWidth()`
- `getSiteLogoHtml()`
- `getSiteNavigationHtml()`

- `getUserInfo()`
- `getUserInfo2()`
- `getWelcomeImageHtml()`

JSON Object Fields

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Shopping methods are implemented with JSON objects. **Set** methods can have JSON objects passed in as input parameter values and **get** methods return strings extracted from JSON objects. Note that parsing of returned JSON values is done when values are set to rows (for example, when cart items are built).

Tables for the following shopping method objects list the fields exposed for each. These fields are for use in web store carts and orders. For convenience, some fields are exposed here that can also be accessed through the existing SuiteScript API.

- [order](#)
- [ach](#)
- [address](#)
- [analyticssettings](#)
- [checkoutsettings](#)
- [creditcard](#)
- [customer](#)
- [discounts_impact](#)
- [discounts](#)
- [fulfillmentPreferences](#)
- [giftcertificate](#)
- [item](#)
- [message](#)
- [orderitem](#)
- [ordersettings](#)
- [ordersummary](#)
- [payment](#)
- [paymentmethod](#)
- [promocode](#)
- [registrationsettings](#)
- [shipmethod](#)
- [sitecategory](#)
- [sitesettings](#)
- [status](#)
- [threedsecure](#)
- [touchpoints](#)

order

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains other objects, and includes the complete set of data for a web store shopping order.

Object Name	Object Type	Description
billaddress	JSON object (type of address)	Billing address for order
customer	JSON object	Current shopper information
customfields	JSON object	List of custom fields name/value pairs set for order
giftcertificate s	Array	Gift certificates applied to order
items	Array of orderitem s	Items added to order
payment	JSON object	Payment information for order
promocode s	Array	Promotion codes applied to order
purchaseNumber	JSON object	Purchase order number for the order
shipaddress	JSON object (type of address)	Shipping address for order
shipmethod	JSON object	Shipping method for order
status	JSON object	Status of order
summary	JSON object (type of ordersummary)	Summary information for order (subtotal, total, tax, etc.)

ach

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains ACH payment information.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
internalid	string	Internal ID of the ACH bank account	yes	yes
mask	string	Masked account number	yes	no
bankname	string	Bank name	yes	no
routingnumber	string	Routing number	yes	no
ownername	string	Name of the owner of the bank account	yes	no
accounttype	string	Type of bank account Value should be Checking or Saving	yes	no
limit	string	Maximum order limit in US dollars	yes	no

address

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains address information for a web store shopping order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
addressee	string	Person to which order is sent	yes	yes (reqd)
addr1	string	First line of address	yes	yes (reqd)
addr2	string	Second line of address	yes	yes
addr3	string	Third line of address	yes	yes
attention	string	Text for ATTN: line of address	yes	yes
city	string	City	yes	yes (reqd)
country	string	Country	yes	yes (reqd)
defaultbilling	string	Indicates whether address is default billing address Value should be T or F.	yes	yes
defaultshipping	string	Indicates whether address is default shipping address Value should be T or F.	yes	yes
internalid	string	Internal ID	yes	yes
isresidential	string	Indicates whether address is residential Value should be T or F.	yes	yes
phone	string	Phone number	yes	yes
		 Note: Address forms on Reference Checkout 2.04 and earlier require the phone number field.		
state	string	State for address	yes	yes (reqd)
zip	string	Zip code	yes	yes (reqd)

analyticssettings

 **Applies to:** SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
clickattributes	string		yes	no
confpagetrackinghtml	string		yes	no
submitattributes	string		yes	no

checkoutsettings

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains information used for checkout.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
cancelurl	string	Full URL path for redirect after cancel action during order submission Used for PayPal Express support	no	yes
continueurl	string	Full URL path for redirect after order submission Used for PayPal Express support	no	yes
createorder	boolean	If value is set to T, backend submits current shopping order when Continue link is clicked Default value is F Used for PayPal Express support	no	yes
custchoosespaymentmethod	string	Value should be T or F.	yes	no
paymentauthorization	threedsecure object with fields	Settings to handle payment authorization Used for 3D Secure support	yes	yes
paymentmandatory	string	Indicates whether payment is required Value should be T or F.	yes	no
paypalexpress	Object with fields	Used for PayPal Express support Includes the following fields: ■ <i>available</i> (boolean) (must be set to T to support PayPal Express) ■ <i>imageurl</i> (string)	yes	no
requiretermsandconditions	string	Indicates whether terms and conditions text is required Value should be T or F.	yes	no
saveccinfo	string	Indicates whether credit card should be saved by default Value should be T or F.	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
shippingaddrfirst	string	Indicates whether shipping address should be displayed first Value should be T or F.	yes	no
showpurchaseorder	string	Indicates whether purchase order should be displayed Value should be T or F.	yes	no
showsavecc	string	Indicates whether the Save Credit Card field should be displayed Value should be T or F.	yes	no
termsandconditions	string	Text of Terms and Conditions field	yes	no
type	string	Used for integration with third party checkout providers Valid values are: paypalexpress, google	no	yes

creditcard

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains credit card information.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
ccdefault	string	Indicates whether credit card is the default Value should be T or F.	yes	yes
ccexpiredate	string	Card expiration date	yes	no
ccname	string	Name on card	yes	yes
ccnumber	string	Card number Value returned by Get function is masked.	yes	yes
ccsecuritycode	string	Card security code Only needed if payment method is creditcard.	no	yes
customercode	string	Customer code	yes	yes
debtcardissueno	string	Card issue number for UK Note: Not exposed in Customer record.	yes	yes

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
expmonth	string	Card expiration month	yes	yes
expyear	string	Card expiration year	yes	yes
internalid	string	Internal ID of credit card. If the Payment Instruments feature is enabled, and the credit card has been tokenized, this contains the internal ID of payment card token.	yes	yes
paymentmethod	Object of type paymentmethod	Payment method type Note: For credit card payments, this is the numerical code of the credit card type. If the Payment Instruments feature is enabled, and the credit card has been tokenized, this is the numerical code of the payment token type.	yes	yes
savecard	string	Indicates whether the card should be saved. Possible values are: ■ F - the card is not saved. ■ T - the card is saved. If the order.submit() method is called with savecard set to T, the card is saved and it is also set as the default card. If the customer methods addCreditCard() and updateCreditCard() are called, the card is saved regardless of the value of the savecard parameter. However, for these methods, the card is set as the default card only if the value of ccdefault is T.	no	yes
validfrom	string	Card valid from date Note: Not exposed in Customer record.	yes	no
validfrommon	string	Card valid from month for UK	yes	yes
validfromyear	string	Card valid from month for UK	yes	yes

customer

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains information about the logged in customer.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
addressbook	List of JSON address objects	Address information	yes	no
balance	string	Customer's accounts receivable balance due, shown in customer's currency.	yes	no
campaignsubscriptions	List of JSON subscription objects	Information about campaign subscriptions for the given customer.	yes	yes
companyname	string	Customer company name	yes (for Checkout and Shopping domains)	yes (for Checkout domain only)
cookieoptions	boolean	Returns a list of cookies for which the customer selected to opt in or out. These are returned only when the SuiteCommerce Analytics Data feature is enabled. For example: "cookieoptions": { "ANALYTICS": false, "PERFORMANCE": true }	yes	yes
creditcards	List of JSON creditcard objects	Credit card information	yes	no
creditholdoverride	string	Customer's credit hold status.	yes	no
creditlimit	string	Maximum currency amount the customer is allowed to accrue in outstanding receivables.	yes	no
email	string	Customer email address Required in Set function for customer registration, guest registration, and login.	no	yes
emailssubscribe	string	Email subscription Value should be T or F.	yes	yes
firstname	string	Customer first name	yes	yes
internalid	string	Internal ID	yes	no
isperson	boolean	If false this customer is of customer type. If true this customer is of the type individual.	yes	no
lastname	string	Customer last name	yes	yes
middlename	string	Customer middle name	yes	yes
name	string	Customer name	yes	yes

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
 Important: As of Version 2013 Release 1, this field is no longer used and is included for backward compatibility only. The firstname, lastname, and middlename fields are the preferred fields for getting and setting customer names.				
partner	string	Partner	yes	yes
password	string	Password Required in Set function for customer registration and login.	no	yes
password hint	string	Hint	no	yes
paymentterms	JSON object of fields	Payment terms associated with the customer. These are returned only if the website sales order type is set to Per Customer Basis.	yes	no
phoneinfo	JSONObject of fields	Returns or sets the phone number information for a customer. For example: {"phone":"65460780", "fax" : "40540", "altphone" : null}	yes	yes

discounts_impact

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains information about the overall discount impact for a given item.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
discounts	Array of JSON object of type discounts	Discounts applied to the order.	yes	no
total_amount	number	Total currency amount of the discount applied to the order.	yes	no
total_amount_formatted	string	Discount amount formatted with currency symbol.	yes	no

discounts

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains information about the per line discount impact for a given item.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
promotion_couponcode	string	The name of the coupon code. This corresponds to what the user applies during the checkout process. For example, user might apply SUMMER or FREESHIP.  Note: If you manually add the discount without using a coupon code, the value of promotion_couponcode is Empty.	yes	no
promotion_name	string	Name of the associated promotion. For example, name of the promotion can be 10% Summer Sale.  Note: If you manually add the discount without using a coupon code, the value of promotion_name is Empty.	yes	no
promotion_id	integer	ID of the associated promotion.  Note: If you manually add the discount without using a coupon code, the value of promotion_id is Null.	yes	no
discount_item	string	Name of the associated discount item. For example, name of the discount item can be Ten Percent.	yes	no
discount_item_id	integer	ID of the associated discount item.	yes	no
amount	number	Currency amount of the discount applied to the order after applying this coupon code.	yes	no
amount_formatted	string	Discount amount formatted with currency symbol.	yes	no
is_auto_applied	boolean	Indicates whether the promocode was automatically or manually applied for the current order. Value should be true or false.	yes	no

fulfillmentPreferences

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains information about the fulfillment choices for a web store shopping order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
fulfillmentChoice	string	Indicates the type of fulfillment, and its value should be ship or pickup . i Note: The value pickup can be used only when the Store Pickup feature is enabled.	yes (reqd)	yes (reqd)
pickupLocationId	integer	Indicates the internal ID of the pickup location, and is available only if the fulfillment choice is pickup .	yes	no
shippingAddressId	integer	Indicates the shipping address ID, and is available only if the fulfillment choice is ship . i Note: This can only be used only when the Multiple Ship To feature is enabled.	yes	no
shippingMethodId	integer	Indicates the shipping method ID, and is available only if the fulfillment choice is ship . i Note: This can only be used only when the Multiple Ship To feature is enabled.	yes	no

giftcertificate

i Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains information about gift certificates in the order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
amountapplied	number	Amount applied to order	yes	no
amountremaining	string	Remaining gift certificate amount after applied to order	yes	no
giftcertcode	string	Gift certificate code	yes	yes
originalamount	number	Original amount of gift certificate	yes	no

item

① Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains basic information about an item that can be included in the order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
featureddescription	string	Featured description	yes	no
internalid	string	Internal ID for item Unique identifier that should be used when adding an item to the cart	yes	yes (reqd)
isavailable	string	Indicates whether item is available for web store order Value should be T or F.	yes	no
isdonationitem	string	Indicates whether item is donation item Value should be T or F.	yes	no
isdropshipitem	string	Indicates whether item is a dropship item Value should be Yes or No.	yes	no
isspecialorderitem	string	Indicates whether item is a special order item Value should be Yes or No.	yes	no
isstorespecial	string	Indicates whether item is on special in the web store Value should be T or F.	yes	no
istaxable	string	Whether the item is taxable Value should be Yes or No.	yes	no
itemtype	string	Type of the item, such as inventory, kit/package, etc.	yes	no
nopricemessage	string		yes	no
nopricemessage2	string		yes	no
outofstockmessage	string		yes	no
outofstockmessage2	string		yes	no
outofstocknobackorder message	string		yes	no
pagetitle	string		yes	no
pagetitle2	string		yes	no
quantityavailable	string	Number of items available for ordering	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
rate	string	Unit price	yes	no
rate_formatted	string	Unit price formatted with currency symbol	yes	no
storedetaileddescription	string	Store detailed description	yes	no
storedescription	string	Store description	yes	no
storedisplayimage	string		yes	no
storedisplayname	string	Store display name	yes	no
storedisplayname2	string	Second store display name	yes	no
storedisplaythumbnail	string		yes	no
freegiftpromotionid	string	ID of the free gift promotion this item belongs to	yes	no

message

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
text	string (with HTML markup)	Text of message		
type	string	Type of message - Confirmation - Error - Information - Warning		

orderitem

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

The orderitem object is a subtype of [item](#).

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
amount	string	Currency amount for items	yes	no
amount_formatted	string	Amount formatted with currency symbol	yes	no
internalid	string	Internal ID for item Unique identifier that should be used when adding an item to the cart	yes	yes (reqd)

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
name	string	Display name	yes	no
options	JSON object	List of name/value pairs	yes	yes
orderid	string	Unique ID for an item in an order	yes	yes
promotionamount	string	Amount after any line-level discounts	yes	no
promotionamount_formatted	string	Amount after any line-level discounts formatted with currency symbol	yes	no
promotiondiscount	string	Amount of discount related to promotions	yes	no
promotiondiscount_formatted	string	Amount of discount related to promotions formatted with currency symbol	yes	no
quantity	string	Number of items in order	yes	yes
rateschedule	string	Price schedule for quantity pricing	yes	no
shipaddress	string	ID of shipping address for item	yes	yes
shipcarrier	string	Shipping carrier	yes	no
		 Note: Valid values that can be set for shipcarrier are ups or nups.		
shipmethod	JSON object of type shipmethod	ID of shipping method for item Used when the Multiple Shipping Routes feature is enabled	yes	yes
taxamount	string	The total tax amount for the line item. Applicable only for tax calculations using SuiteTax. Tax calculation using SuiteTax is available only for US and Canada.	yes	no
taxdetailsreference	string	The ID that links the tax details returned by <code>getTaxDetails()</code> with the item to which they apply. Applicable only for tax calculations using SuiteTax. Tax calculation using SuiteTax is available only for US and Canada.	yes	no
taxtype1	string	Type of the item tax. For example, GST/VAT/WET.	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
		Not applicable if SuiteTax is enabled.		
taxrate1	string	Rate of the tax in percentage.	yes	no
		Not applicable if SuiteTax is enabled.		
taxtype2	string	For Canada only. Currently returns PST.	yes	no
		Not applicable if SuiteTax is enabled.		
taxrate2	string	For Canada only. The rate of PST tax in percentage when applicable.	yes	no
		Not applicable if SuiteTax is enabled.		
tax1amt	string	The total tax amount for the line item.	yes	no
		Not applicable if SuiteTax is enabled.		
fulfillmentPreferences	JSON object of type fulfillmentPreferences	Fulfillment preference for item	yes	no
discounts_impact	JSON object of type discounts_impact	Per line discount impact for the line item.	yes	no

ordersettings

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains general settings for the order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
itemdisplayorder	string	Ordering of items in an order	yes	no
outofstockbehavior	string	Behavior for out of stock items	yes	no
upselldisplay	string	■ Related Items Only ■ Upsell Items ■ Related Items and then Upsell Items ■ Upsell Items and then Related Items	yes	no

ordersummary

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains summary information for the order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
discountedsubtotal	number	Item subtotal - any applicable discount	yes	no
discountedsubtotal_formatted	string	Item subtotal - any applicable discount formatted with currency symbol	yes	no
discountrate	string	Raw discount	yes	no
discounttotal	number	Total amount of discounts applied	yes	no
discounttotal_formatted	string	Total amount of discounts applied formatted with currency symbol	yes	no
giftcertapplied	number	Total amount of gift certificates applied	yes	no
giftcertapplied_formatted	string	Total amount of gift certificates applied formatted with currency symbol	yes	no
handlingcost	number	Amount of handling cost	yes	no
handlingcost_formatted	string	Amount of handling cost formatted with currency symbol	yes	no
itemcount	number	 Important: Use this argument with the getOrderSummary method for the most optimized call. Note that the value returned equals the total quantity of items in the cart which may differ from the total lines in the cart. For example, if a shopper has three of the same items in the cart, the call returns 3.	yes	no
shippingcost	number	Amount of shipping cost	yes	no
shippingcost_formatted	string	Amount of shipping cost formatted with currency symbol	yes	no
subtotal	number	Total of quantity * rate per line item	yes	no
subtotal_formatted	string	Total of quantity * rate per line item formatted with currency symbol	yes	no
tax2total	number	Total secondary tax on taxable elements of the order (items + shipping + handling) - effect of any discounts. Not applicable if SuiteTax is enabled.	yes	no
tax2total_formatted	string	Total secondary tax on taxable elements of the order (items + shipping + handling) - effect of any discounts formatted with currency symbol. Not applicable if SuiteTax is enabled.	yes	no
taxondiscount	number	For VAT countries, currency amount of the tax rate * the discount amount	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
taxondiscount_formatted	string	For VAT countries, currency amount of the tax rate * the discount amount formatted with currency symbol	yes	no
taxonhandling	number	Currency amount of the tax rate * taxable handling amount	yes	no
taxonhandling_formatted	string	Currency amount of the tax rate * taxable handling amount formatted with currency symbol	yes	no
taxonshipping	number	Currency amount of the tax rate * taxable shipping amount	yes	no
taxonshipping_formatted	string	Currency amount of the tax rate * taxable shipping amount formatted with currency symbol	yes	no
taxtotal	number	Total tax on taxable elements of the order (items + shipping + handling) - effect of any discounts	yes	no
taxtotal_formatted	string	Total tax on taxable elements of the order (items + shipping + handling) - effect of any discounts formatted with currency symbol	yes	no
totalcombinedtaxes	number	A grand total of all the tax components (taxtotal + tax2total) associated with an order	yes	no
totalcombinedtaxes_formatted	string	A grand total of all the tax components (taxtotal + tax2total) associated with an order formatted with currency symbol	yes	no
total	number	Order subtotal + shipping, handling, and all tax, less the effect of any discounts	yes	no
total_formatted	string	Order subtotal + shipping, handling, and all tax, less the effect of any discounts formatted with currency symbol	yes	no

payment

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains payment information for the order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
ach	JSON ach object	Payment information when ach is selected as the payment method Note: See Formatting Values for ACH Payments for an example of how to format this information.	yes	yes

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
creditcard	JSON creditcard object	Payment information when credit card is selected as the payment method Note: See Formatting Values for Credit Card Payments for an example of how to format this information. If the Payment Instruments feature is enabled, and the credit card has been tokenized, this field contains the internal ID and payment method of payment card token.	yes	yes
paymentmethod	JSON paymentmethod object	Details of the payment method selected by customer.	yes	yes
paymentterms	string	Payment terms selected by customer. Value should be CreditCard, Ach, or Invoice. Notes: Set this value to Invoice to use an invoice as payment. See Using an Invoice as Payment for an example.	yes	yes
paypal	JSON object	PayPal information if enabled	yes	no
status	JSON status object	Embedded status object used for web store business logic validation	yes	yes

Formatting Values for ACH Payments

The following example code illustrates how to format details for a payment using an existing ACH bank account:

```

1 function service(request,response)
2 {
3
4   payment = {
5     'paymentterms' : 'Ach',
6     'ach': {
7       'internalid': '146'
8     }
9   };
10
11
12   var order = nlapiGetWebContainer().getShoppingSession().getOrder();
13   order.setPayment(payment)
14
15   };

```

Formatting Values for Credit Card Payments

The following example code illustrates how to format details for a payment using a new credit card:

```

1 function service(request,response)
2 {
3
4   var payment = {
5     creditcard: {
6       'ccname': 'Herb Joseph',
7       'ccnumber': '4111111111111111',
8       'expmonth': '09',
9       'expyear': '2013',
10      'paymentmethod': {
11        'internalid': '7',
12        'name': 'Visa',
13        'ispaypal': 'F',
14        'creditcard': 'T',
15        'key': '8,8,1135464'}
16      }
17    };
18
19    var order = nlapiGetWebContainer().getShoppingSession().getOrder();
20    order.setPayment(payment)
21
22  };

```

The following example code illustrates how to format details for a payment using a stored credit card:

```

1 function service(request,response)
2 {
3
4   var payment = {
5     creditcard: {
6       'internalid': '35',
7       'paymentmethod': {
8         'internalid': '7',
9         'name': 'Visa',
10        'ispaypal': 'F',
11        'creditcard': 'T',
12        'key': '8,8,1135464'}
13      }
14    };
15
16    var order = nlapiGetWebContainer().getShoppingSession().getOrder();
17    order.setPayment(payment)
18
19  };

```

Using an Invoice as Payment

The following example code illustrates how to use an invoice as a payment method:

```

1 function service(request,response)
2 {
3     var returnval = null;
4
5     try
6     {
7         var payment = {
8             paymentterms : 'Invoice'
9         };
10        nlapiGetWebContainer().getShoppingSession().getOrder().setPayment(payment);
11    }
12    catch (e)
13    {
14        var nle = nlapiCreateError(e);
15        returnval = {status : 'error', reasoncode : nle.getCode(), message : nle.getDetails()};
16    }
17    response.writeLine(JSON.stringify(returnval));
18 }

```

paymentmethod

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object includes payment method information for the order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
internalid	string	Internal ID	yes	no
creditcard	string	Indicates whether payment method is a credit card Value should be T or F. If the Payments Instruments feature is enabled, this contains T if the payment method is a payment card token.	yes	no
creditcardtoken	string	Indicates whether payment method is a payment card token. Value should be T or F. Applicable only if the Payments Instruments feature is enabled.	yes	no
ispaypal	string	Indicates whether payment method is PayPal Value should be T or F.	yes	no
name	string	Name of the payment method	yes	no
paypalemailaddress	string	Primary paypal email address	yes	no
merchantid	string	Internal ID of the merchant account linked with the payment method	yes	no
isexternal	string	Indicates whether payment method is handled outside of NetSuite Value should be T or F	yes	no
imagesrc	string	Returns the URL of the image file used to represent the payment method.	yes	no
isautomatedclearinghouse	string	Indicates whether payment method is ACH. Value should be T or F	yes	no

Note: You cannot set an invoice as a payment method. Instead, set invoice as the paymentterms value for the [payment](#) object. See [Using an Invoice as Payment](#) for an example.

promocode

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains promotion code information for the order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
discountamount	string	Currency amount of the discount applied to the order.	yes	no
errormsg	string	Returns the error message when a promo code is not valid.	yes	no
internalid	string	Internal ID	yes	no
isvalid	string	Indicates whether promocode is valid for the current order. Value should be T or F.	yes	no
promocode	string	The value of the Coupon Code ID. This corresponds to what the user sets during the checkout process.	yes	yes
promocodeid	string	The value of the Promocode ID. This corresponds to the promocode displayed in the NetSuite UI. Note: The value of this field is not changed when a coupon code is modified.	yes	yes
purchase_discount_amount	string	Currency amount of the discount applied to the order.	yes	no
purchase_discount_amount_formatted	string	Discount amount formatted with currency symbol.	yes	no
shipping_discount_amount	string	Currency amount of the discount applied to the shipping cost.	yes	no
shipping_discount_amount_formatted	string	Discount shipped amount formatted with currency symbol.	yes	no
promotion_name	string	Name of the promotion associated with the applied coupon.	yes	no
discount_name	string	Name of the discount associated with the applied promotion.	yes	no
discount_type	string	Indicates the type of discount, and its value should be PERCENTAGE or FLAT.	yes	no
discount_rate	string	Rate of the discount in percentage if the type of discount is PERCENTAGE and currency amount of the discount if the type of discount is FLAT.	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
is_auto_applied	boolean	Indicates whether the promocode was automatically or manually applied for the current order. Value should be true or false.	yes	no
applicability_status	string	Indicates whether the promocode is applied to the current order or not. Value should be one of the following: ■ APPLIED — Indicates that the promocode is applied to the current order and you get a discount if the promocode is valid. It also indicates that the type of promotion is SuitePromotion. ■ NOT_APPLIED — Indicates that the promocode does not give you a discount even though it is added to the current order. It also indicates that the type of promotion is SuitePromotion. ■ NOT_AVAILABLE — Indicates that the applicability status information is not available. You get this value in case of Standard promotions.	yes	no
applicability_reason	string	Specifies the reason for not applying the promocode. Value should be one of the following: ■ DISCARDED_BEST_OFFER — Indicates that the promocode is not applied as there is another promocode or a combination of promocodes that is giving you a better discount. ■ CRITERIA_NOT_MET — Indicates that the promocode is not applied as it does not meet the criteria. For example, if your order does not meet the minimum order amount requirement. ■ NOT_AVAILABLE — Indicates that the promocode is applied or the promotion is a Standard promotion.	yes	no
promotion_type	string	Indicates the type of promotion.	yes	no

registrationsettings

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
companyfieldmandatory	string	Indicates whether Company field is required Value should be T or F.	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
registrationallowed	string	Indicates whether registration is allowed Value should be T or F.	yes	no
registrationanonymous	string	Indicates whether registration is anonymous Value should be T or F.	yes	no
registrationmandatory	string	Indicates whether registration is required Value should be T or F.	yes	no
registrationoptional	string	Indicates whether registration is optional Value should be T or F.	yes	no
showcompanyfield	string	Indicates whether to show Company field on registration form	yes	no

shipmethod

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains shipping method information for the order.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
name	string	Name of the shipping method	yes	no
rate	string	Rate of the shipping method	yes	no
shipcarrier	string	Shipping carrier ID	yes	yes
 Note: Valid values that can be set for shipcarrier are ups or nups.				
shipmethod	string	Shipping method ID	yes	yes

sitecategory

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
canonicalurl	string		yes	no
categorythumbnailhtml	string		yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
description2	string	The value from the web site category record for the field Brief Description.	yes	no
internalid	string	Internal ID of the site category.	yes	no
itemfields	string	An array of field names returned for each category.	yes	no
itemid	string	Name of site category	yes	no
metataghtml	string	The value from the web site category record for the field Meta Tag Html.	yes	no
pagetitle	string		yes	no
pagetitle2	string		yes	no
searchkeywords	string	The value from the web site category record for the field Search Keywords.	yes	no
storedetaileddescription	string		yes	no
storedetaileddescription2	string		yes	no
storedisplayimage	string		yes	no
storedisplayimagehtml	string		yes	no
storedisplaythumbnail	string		yes	no
storeurl	string		yes	no
urlcomponent	string		yes	no
urlcomponentaliases	array		yes	no
welcomepagetitle	string		yes	no

sitesettings

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
analytics	JSON analyticssettings object	Analytics settings	yes	no
checkout	JSON checkoutsettings object	Checkout settings	yes	no
cookiepolicy	string	Returns the URL for the cookie policy file in the NetSuite file cabinet.	yes	no
currencies	Array of JSON objects with fields (internalid, name, isdefault, symbolplacement)	Ordered list of currencies supported by the site isdefault field is a Boolean, with a value of T for default currency, F for others.	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
		symbolplacement uses the constants: SYMBOL_BEFORE_NUMBER = 1 and SYMBOL_AFTER_NUMBER = 2		
defaultshipcountry	string	Default ship to country	yes	no
defaultshippingmethod	string	Default shipping method	yes	no
displayname	string	Site name	yes	no
imagesizes	Array of JSON objects with image resize definitions (resizeid, resizeh, resizew)	Each object has the following properties: - maxwidth-maximum resized image width in pixels - maxheight-maximum resized image height in pixels - urlsuffix-suffix that should be added to the image URL to return a resized image - internalid-internal system ID of image resize definition. - name-value from the Image Resize ID field (image resize definition) as entered on the Web Site Setup page.	yes	yes
iswebstoreoffline	boolean	Retruns T or F depending on whether the web store is online or offline.	yes	no
iswsdk	boolean	Returns T or F depending on the settings of the WSDK checkbox on the site record (tab setup) and the WSDK feature settings for a site.	yes	no
languages	Array of JSON objects with fields (internalid, name, isdefault)	Ordered list of languages for the site isdefault field is a Boolean, with a value of T for default language, F for others	yes	no
loginallowed	string	Indicates whether login is allowed Value should be T or F.	yes	no
loginrequired	string	Indicates whether login is required to access the site Value should be T or F.	yes	no
order	JSON ordersettings object	Order settings	yes	no
paymentmethods	array of paymentmethod objects	Payment methods	yes	no
pricesincludevat	string	Indicates whether prices include VAT Value should be T or F.	yes	no
registration	JSON registrationsettings object	Registration settings	yes	no
requireloginforpricing	string	Indicates whether login is required to display item prices Value should be T or F.	yes	no
requireshippinginformation	boolean	Indicates whether shipping information needs to be collected Value should be T or F.	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
shipallcountries	string	Indicates whether shipping is supported to all countries Value should be T or F.	yes	no
shippingrequired	string	Indicates whether shipping is required for the site Value should be T or F.	yes	no
shiptocountries	Array of strings	Countries where shipping is supported	yes	no
showextendedcart	string	Indicates whether extended cart should be displayed Value should be T or F.	yes	no
showshippingestimator	string	Indicates whether shipping estimator should be displayed Value should be T or F.	yes	no
sitetype	string	Indicates the type of web site. Value should be ADVANCED or STANDARD. ADVANCED indicates the site is a SuiteCommerce, SuiteCommerce Advanced or SuiteCommerce InStore site whereas STANDARD indicates it is a web site built with Site Builder.	yes	no
subsidiaries	Array of JSON objects with fields (internalid, name, isdefault)	Ordered list of subsidiaries for the site isdefault field is a Boolean, with a value of T for default subsidiary, F for others	yes	
wsdkcancellcarturl	string	For use with an External Catalog Site (WSDK). Returns the Cancel Cart URL.	yes	no
wsdkcancellcheckouturl	string	For use with an External Catalog Site (WSDK). Returns the Cancel Checkout URL.	yes	no
wsdkcancellloginurl	string	For use with an External Catalog Site (WSDK). Returns the Cancel Login URL.	yes	no
wsdkcompletecheckouturl	string	For use with an External Catalog Site (WSDK). Returns the Complete Checkout URL.	yes	no
wsdkcompletelloginurl	string	For use with an External Catalog Site (WSDK). Returns the Complete Login URL.	yes	no

status

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Each JSON object returned by a shopping function has an embedded status object that provides business logic validation of the web store order. The caller of a shopping function, in addition to checking the request status, should also check the status object to determine whether the object can be submitted to the backend and whether there are messages to display.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
code	string	Code of the status Following values are supported: - error = object contains error - incomplete = object is not complete, so cannot be handled by backend - ready = object is complete and ready to be submitted to backend - success = object successfully processed by backend		
fieldstatuses	array of status objects	Status for object fields		
messages	array of message objects	Status messages		
reasoncode	string	Code of possible reason for the current status		

subscriptions

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains information used for campaign subscriptions.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
internalid	string	The internalid of the campaign subscription.	yes	no
name	string	Name of the customer.	yes	no
subscribed	boolean	Indicates whether the customer is currently subscribed to this email campaign category.	yes	yes
description	string	Description of the campaign.	yes	no

summarytax

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains a breakdown of the different types of taxes applicable to a web store shopping order. For example, in Canada, it would contain the total amount of PST and GST. The `summarytax` object is used only with tax calculations using SuiteTax. Tax calculation using SuiteTax is available only for US and Canada.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
taxTypeId	string	The ID of the type of tax.	yes	no
taxTypeName	string	The name of the tax type. For example, PST or GST.	yes	no
taxTotal	number	The total tax amount for the tax type used on the sales order.	yes	no

taxdetails

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains tax information for each line item in a web store shopping order. Each taxdetails object contains details for one type of tax, for example PST. The taxdetails object is used only with tax calculations using SuiteTax. Tax calculation using SuiteTax is available only for US and Canada.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
taxdetailsreference	string	The ID that links the tax details in the taxdetails object with the orderitem to which they apply. The taxdetails object for shipping has the taxdetailsreference SHIPPING. If handling is set up as separate from shipping, then the taxdetails object for handling has the taxdetailsreference HANDLING. Applicable only for tax calculations using SuiteTax. Tax calculation using SuiteTax is available only for US and Canada.	yes	no
linetype	string	The type of line item in the sales order to which the tax applies. For example, 'Item', 'Shipping', and 'Handling'.	yes	no
linename	string	The name of the line item in the sales order to which the tax applies. For items, this returns the name of the item from the item record, for example "Blue Shirt". For shipping, this returns the name of the shipping record plus the term "Shipping Cost", for example "US_CA_Ship_Item_1, Shipping Cost". If handling is set up as separate from shipping, this returns the name of the handling record plus the term "Handling Cost", for example "US_CA_Ship_Item_1, Handling Cost".	yes	no
netamount	number	The cost of the line item, not including tax.	yes	no

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
grossamount	number	The cost of the line item, including tax.	yes	no
taxtype	string	The type of tax. For example, if the type of tax applicable is PST, this contains the ID of the corresponding tax type record.	yes	no
taxcode	string	The tax code associated with the tax type.	yes	no
taxbasis	string	The amount based on which tax is calculated.	yes	no
taxrate	number	The rate of tax being charged.	yes	no
taxamount	number	The total tax amount for the line item for the tax type returned in taxtype.	yes	no
calcdetail	string	The information returned by the tax engine used to calculate the tax.	yes	no

threedsecure

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

This object contains information used for 3D Secure payment integration.

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
noredirect	string	Default is F If set to T, when 3D Secure authorization is required, instead of a redirect, a status object that includes fields for 3D secure integration is returned.	no	yes
servicehtml	string	Complete HTML for IFrame that triggers 3D Secure service	yes	no
termurl	string	URL for page or handler where 3D Secure POST is sent	no	yes
threedsecurekey	string	Key used by backend to identify currenct 3D Secure data object	yes	yes

touchpoints

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

Each field in this object stores the URL for a custom page that is integrated to the web store at a selected entry point. These URLs are defined on the Touch Points subtab of the web application record. See the help topic [Select Supported Touch Points](#).

Field Name	Field Type	Description	In Get Functions?	In Set Functions?
checkout	string	URL for starting the checkout process	yes	no
continueshopping	string	URL for continue shopping page	yes	no
customercenter	string	URL for customer center	yes	no
home	string	URL for home page	yes	no
login	string	URL for customer login	yes	no
logout	string	URL for customer logout	yes	no
register	string	URL for customer registration	yes	no
serversync	string	URL for sync shopper status between shopping and checkout domain	yes	no
viewcart	string	URL for viewing the shopping cart	yes	no
welcome	string	URL for welcome page	yes	no

Serversync Touchpoint

i Applies to: SuiteCommerce Advanced

This information refers to the Mont Blanc release of SuiteCommerce Advanced and later.

You can customize your implementation of SuiteCommerce Advanced to use the serversync touchpoint. This touchpoint maintains the identity of the user and cart contents when a user opens items in other tabs or windows. This is useful during email marketing campaigns, when users open a separate instance of the application from a link in an email.

Using the serversync touchpoint requires customizing your implementation of SuiteCommerce Advanced to extend or override the following SuiteCommerce Advanced source files:

- CheckoutApplication@X.X.X\SuiteScript\checkout.environment.ssp
- Header@X.X.X\JavaScript\Header.Profile.View.js
- ProductList@X.X.X\SuiteScript\ProductList.Model.js
- Profile@X.X.X\SuiteScript\Profile.Model.js



Important: Making changes to core JavaScript source files or changing any vital functionality of the application can make migrating to future releases difficult. Before making changes to SuiteCommerce Advanced, see the help topics [Develop Your SCA Customization](#) and [Best Practices for Customizing SCA](#)

In the case of the serversync touchpoint, NetSuite has provided a customization patch that you can use.



Important: If you have already made customizations to your implementation of SuiteCommerce Advanced, use caution when applying these custom modules.

To apply the serversync touchpoint custom modules:

1. Click the link to download the serversync zip file.
[Serversync.zip](#)
2. Extract the files, then place the Serversync directory and all contents in the Modules > extensions directory for your implementation of SuiteCommerce Advanced.

 **Note:** If you have already made customizations to the checkout.environment.ssp SuiteScript file in the CheckoutApplication@X.X.X module, run a diff between the new custom version of checkout.environment.ssp and your existing override, incorporate NetSuite's customizations into your custom module, and remove the EnvironmentCustom@1.0.0 folder from your new Serversync directory.

Customizations you have made to the Header, ProductList, and Profile modules are only affected if you have modified the same methods and properties that have been extended in this patch. Run a diff on the appropriate files and, if you have made customizations in the same methods and properties, incorporate NetSuite's changes into your custom module.

3. Go to Modules > extensions > Serversync to locate the distro.json provided with the patch.
4. Open the new distro.json file and locate the appropriate Serversync entries for CheckoutApplication, Header, ProductList, and Profile.
5. Copy the Serversync entries and paste them into the list of modules in your existing distro.json file.

 **Important:** If you have already made any customizations to the CheckoutApplication module, or customizations to the same methods and properties of the Header, ProductList, or Profile modules in your implementation of SuiteCommerce Advanced, do not use the Serversync entries from the new distro.json.

6. Open a command prompt from your source directory and deploy to NetSuite.

 **Note:** Since these customizations modify files stored as SSP libraries, changes are not visible in your local environment. You must deploy your custom modules directly to NetSuite. See the help topic [Deploy SCA Customizations to NetSuite](#) for more information.

SSP Application Governance

Applies to: SuiteCommerce | SuiteCommerce Advanced | SuiteCommerce MyAccount

SSP applications have a governance model in accordance with SuiteScript governance model, with the same goal of optimizing performance. SSP application governance is based on usage units. Each shopping method consumes a system-defined number of processing units, and each SSP application script can execute a system-defined number of units. If the number of allowable units is exceeded by an SSP application script, the script is terminated.

The limit for SSP applications is 1000 units per script.

The units consumed per method are as follows:

- Shopping Session Methods
 - `nlobjShoppingSessionImpl.login(20)`
 - `nlobjShoppingSessionImpl.registerCustomer(20)`
 - `nlobjShoppingSessionImpl.registerGuest(20)`
 - `nlobjShoppingSessionImpl.setShopperCurrency(10)`
 - `nlobjShoppingSessionImpl.setShopperLanguageLocale(10)`
 - `nlobjShoppingSessionImpl.setShopperSubsidiary(10)`
 - `nlobjShoppingSessionImpl.getPaymentMethods(10)`
 - `nlobjShoppingSessionImpl.getCountries(10)`
 - `nlobjShoppingSessionImpl.getShipToCountries(10)`
 - `nlobjShoppingSessionImpl.getStates(10)`
 - `nlobjShoppingSessionImpl.getItemFieldValues(10)`
 - `nlobjShoppingSessionImpl.getSiteCategoryFieldValues(10)`
 - `nlobjShoppingSessionImpl.getInformationItemFieldValues(10)`
 - `nlobjShoppingSessionImpl.getMediaItemFieldValues(10)`
- Customer Methods
 - `nlobjWebStoreCustomerImpl.addAddress(10)`
 - `nlobjWebStoreCustomerImpl.updateAddress(10)`
 - `nlobjWebStoreCustomerImpl.removeAddress(10)`
 - `nlobjWebStoreCustomerImpl.addCreditCard(10)`
 - `nlobjWebStoreCustomerImpl.updateCreditCard(10)`
 - `nlobjWebStoreCustomerImpl.removeCreditCard(10)`
 - `nlobjWebStoreCustomerImpl.getAddressBook(10)`
 - `nlobjWebStoreCustomerImpl.getAddress(10)`
 - `nlobjWebStoreCustomerImpl.getCreditCards(10)`
 - `nlobjWebStoreCustomerImpl.getCreditCard(10)`
- Order Methods
 - `nlobjWebStoreOrderImpl.addItem(20)`
 - `nlobjWebStoreOrderImpl.addItem(10)`
 - `nlobjWebStoreOrderImpl.removeItem(10)`

- `nlobjWebStoreOrderImpl.updateItemQuantity(10)`
- `nlobjWebStoreOrderImpl.updateItemQuantities(20)`
- `nlobjWebStoreOrderImpl.removeAllItems(10)`
- `nlobjWebStoreOrderImpl.estimateShippingCost(20)`
- `nlobjWebStoreOrderImpl.applyPromotionCode(5)`
- `nlobjWebStoreOrderImpl.removePromotionCode(5)`
- `nlobjWebStoreOrderImpl.applyGiftCertificate(5)`
- `nlobjWebStoreOrderImpl.removeAllGiftCertificates(5)`
- `nlobjWebStoreOrderImpl.setShippingAddress(10)`
- `nlobjWebStoreOrderImpl.setShippingMethod(5)`
- `nlobjWebStoreOrderImpl.getAvailableShippingMethods(20)`
- `nlobjWebStoreOrderImpl.getAvailableShippingMethod(10)`
- `nlobjWebStoreOrderImpl.submit(20)`



Note: There is no concurrency limit for SSP applications but you should not use them for high volume catalog pages or to deliver content that should be delivered from a cache.